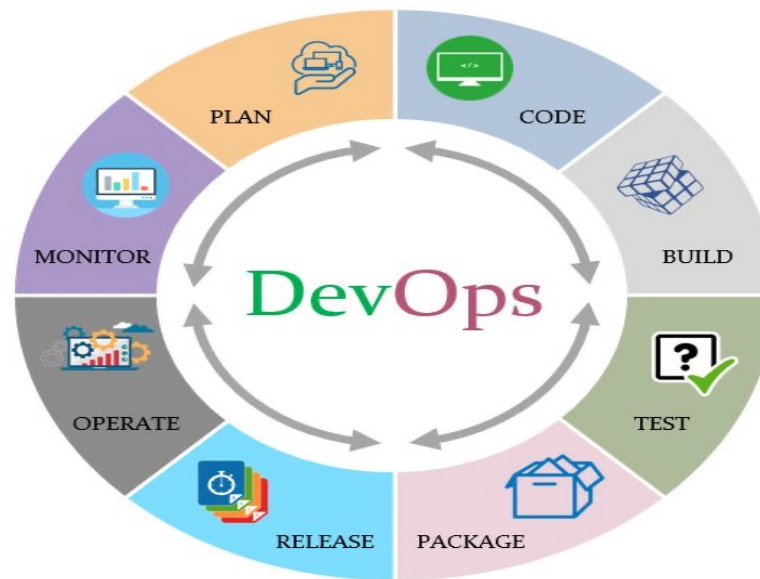
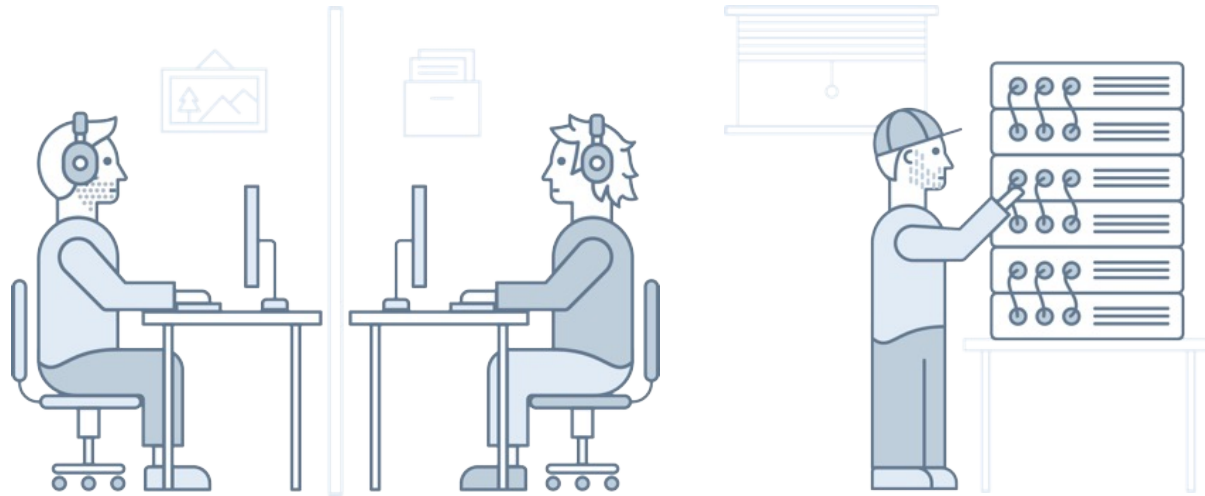


Dr. Oscar Sanjuán.

Repaso de la Asignatura



Dr. Oscar Sanjuán.



Tema 1:

¿Qué es DevOps?

Universidad Internacional de La Rioja

DevOps (Dev + Ops)

¿QUÉ ES?

DevOps es la combinación de filosofías, prácticas y herramientas culturales que **aumenta la capacidad de una organización para entregar aplicaciones y servicios a alta velocidad:** evolucionando y mejorando sus productos a un ritmo más rápido que las organizaciones que utilizan procesos tradicionales de desarrollo de software y gestión de la infraestructura. Esta velocidad permite a las organizaciones ofrecer un mejor servicio a sus clientes y competir de una manera más agresiva en el mercado.

PRINCIPIOS DevOps

Colaboración

Responsabilidad compartida entre los miembros de extremo a extremo

Mejora continua

Automatización

Foco en el cliente

Aprendizaje de los errores

Equipos multifuncionales

METODOLOGÍAS

Integración continua

Entrega continua

Despliegue continuo

Gestión de la configuración

Monitorización

PRINCIPALES MITOS

DevOps solo es válido para empresas nacidas en la web

DevOps es operaciones aprendiendo a codificar

DevOps solo involucra a desarrollo y operaciones

DevOps no es válido para ITIL

DevOps implica únicamente un cambio en la comunicación

DevOps no es apto para las industrias reguladas

DevOps no es aplicable a equipos de *outsourcing*

Sin nube, no hay DevOps

¿A qué llamamos DevOps?

- **DevOps** es un término relativamente nuevo para describir lo que también ha sido llamado como **administración de sistemas ágiles**. DevOps se centra en el trabajo y la colaboración de los equipos de desarrollo y operaciones.

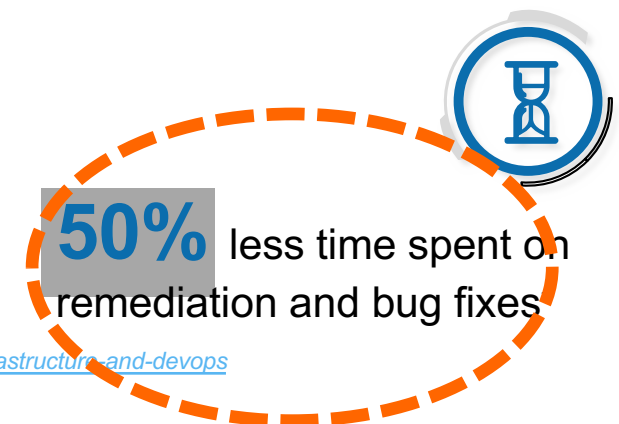
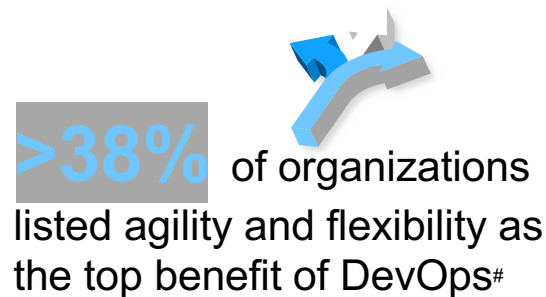
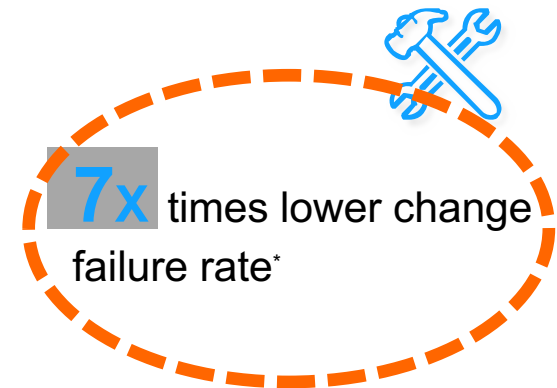
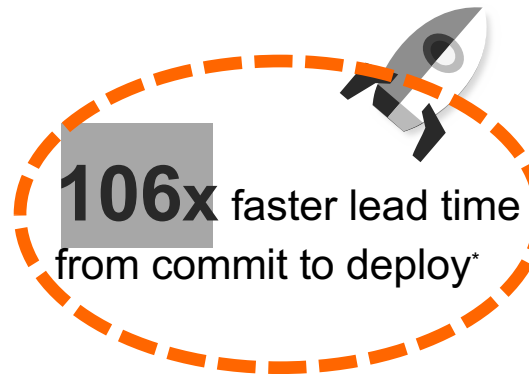
Pero... es esto DevOps para todos???

- ..
- ..
- ..



Beneficios típicos de DevOps

Test coverage is critical to achieve many of the DevOps benefits.



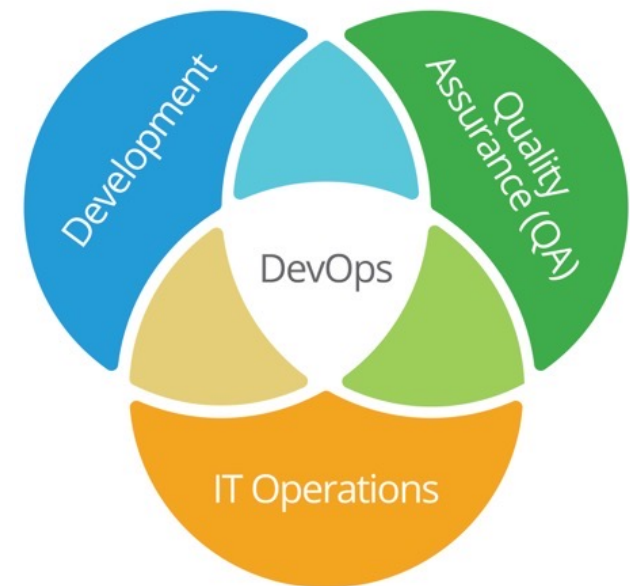
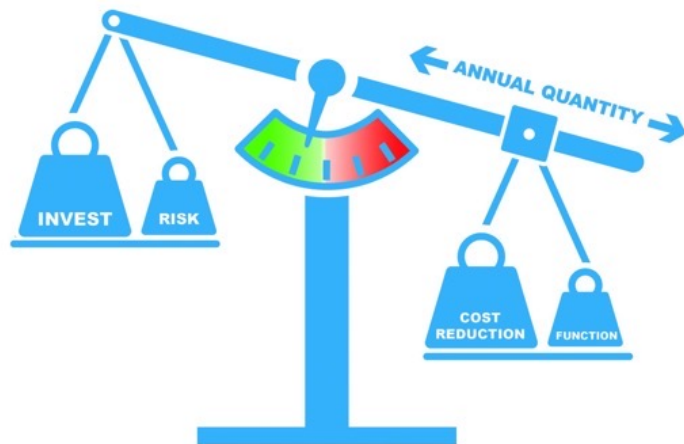
*: <https://services.google.com/fh/files/misc/state-of-devops-2019.pdf>

\$: <http://www.compaid.co.in/Articles/DevOps-Value-and-Cost-Savings>

#: <https://451research.com/about-us/our-research/research-channels/applied-infrastructure-and-devops>

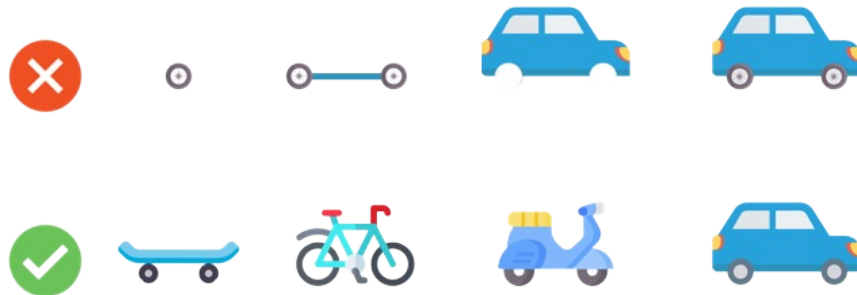
Principios DevOps (I)

- Fomentar un ambiente de trabajo colaborativo
 - Shared Code Ownership.
- Establecer una responsabilidad de extremo a extremo
 - Terminado significa que esta en manos del cliente.
- Fomentar la mejora continua
 - Todos podemos opinar y mejorar los procesos.



Principios DevOps (II)

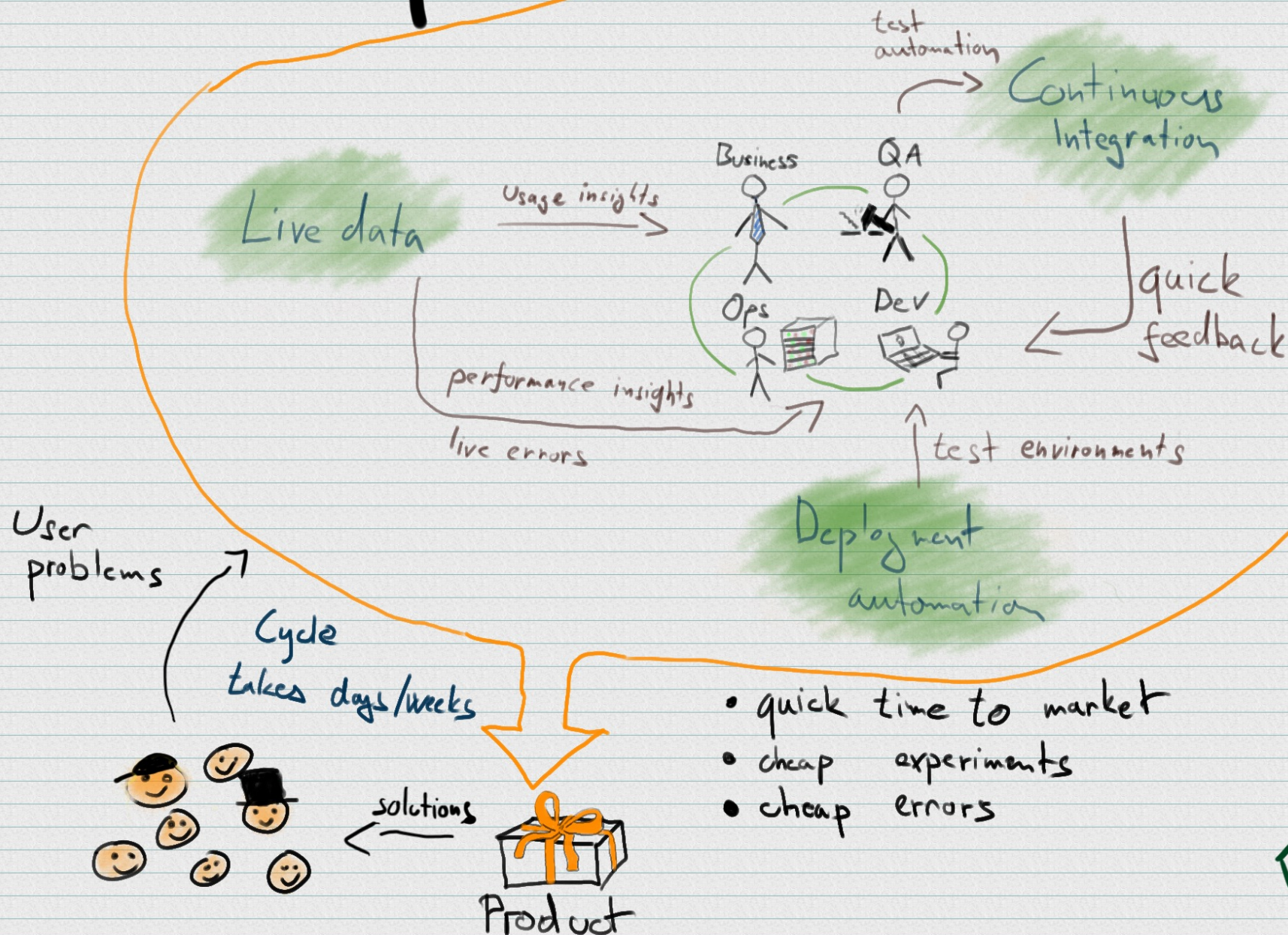
- Fomentar la automatización
 - Automatizar permite dedicarnos a lo importante.
- Dar prioridad a las necesidades del cliente
 - "Release Value"
- Aceptar el fracaso y aprender de él
 - Try fast Fail Fast
- Formar equipos multifuncionales
 - Feature Teams



Metodologías y estrategias DevOps

- Las metodologías de DevOps incluyen:
- Integración continua, que incluye a la creación de código, la construcción, la integración y las pruebas.
- Entrega continua, que incluye a la integración continua, pero se centra en las entregas.
- Despliegue continuo, que se centra en automatizar las entregas de la forma más rápida posible.
- Gestión de la configuración y monitorización continua.

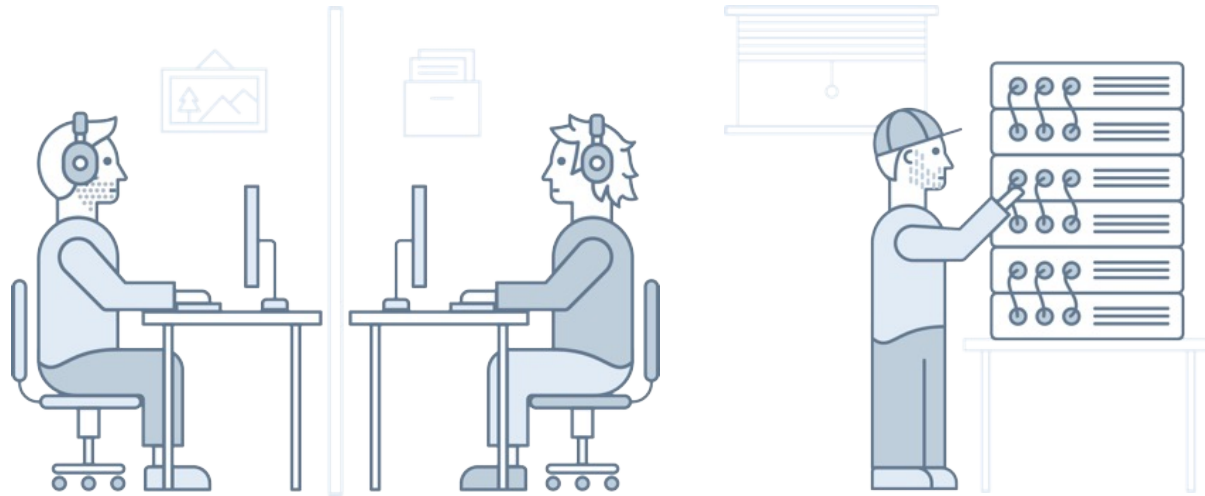
DevOps



- quick time to market
- cheap experiments
- cheap errors

Filip Sedláček
2017

Dr. Oscar Sanjuán.



Tema 2: Cultura de la Colaboración

Universidad Internacional de La Rioja

COLABORACIÓN

CAMBIO CULTURAL

Una cultura DevOps se caracteriza por un alto grado de colaboración a través de los roles, centrado en el bienestar y los objetivos del negocio, la confianza y el aprendizaje a través de la experiencia.

VÍAS HACIA LA CREACIÓN DE UNA CULTURA DEVOPS

- Identificación de los objetivos de negocio
- Medición de la cultura
- Creación de un ambiente de intercambio
- Formación de un equipo DevOps
- Diseño de procesos DevOps

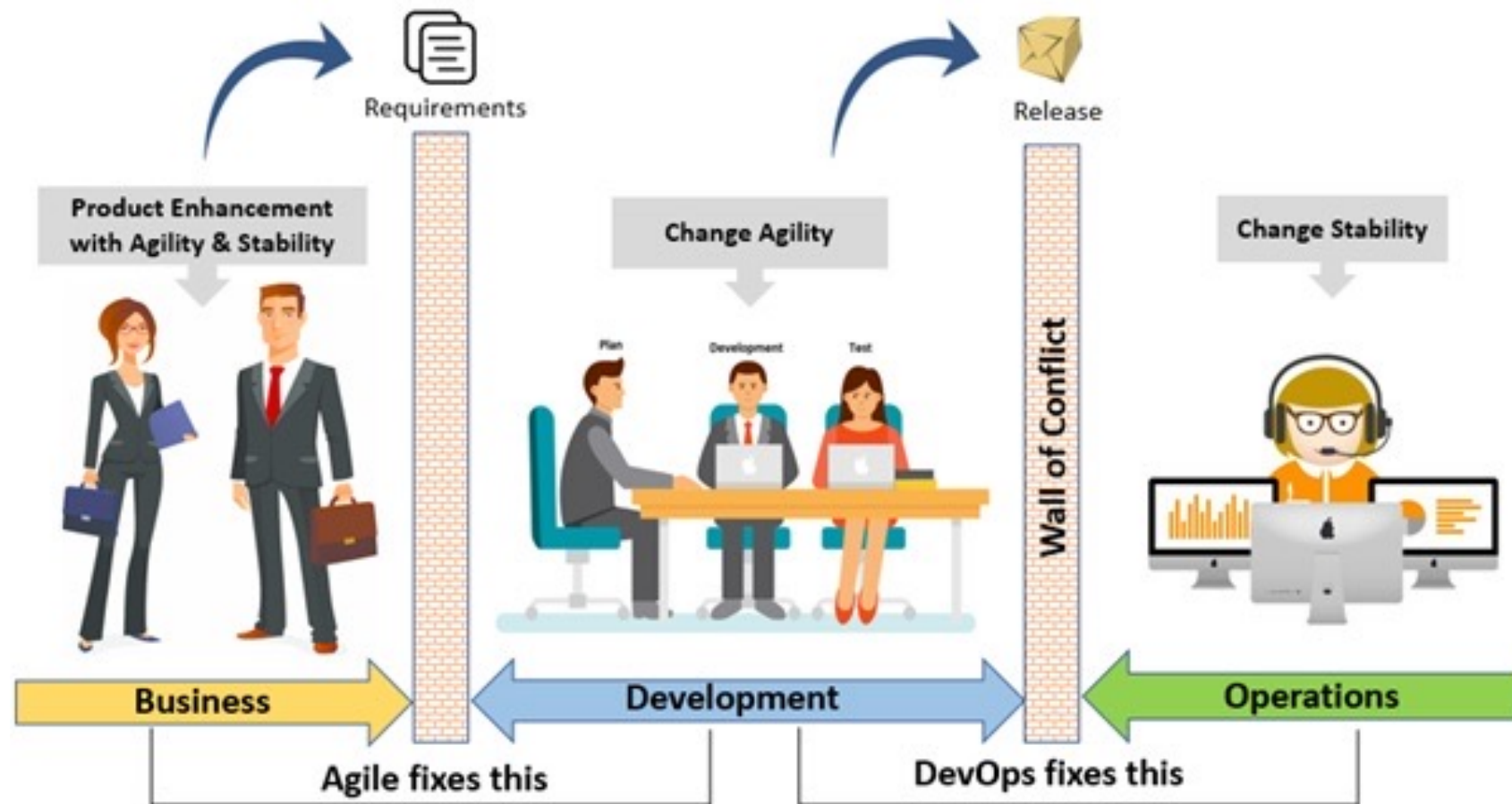
HERRAMIENTAS

Metodológicas (Metodologías ágiles)

- *Adaptative Software Development* (ASD)
- Crystal Clear
- *Feature Driven Development* (FDD)
- Kanban
- *Extreme Programming* (XP)
- Scrum

De software

- Slack
- Teams
- Basecamp
- Dropbox
- Trello
- Office 365



Factores Clave para el Cambio Cultural

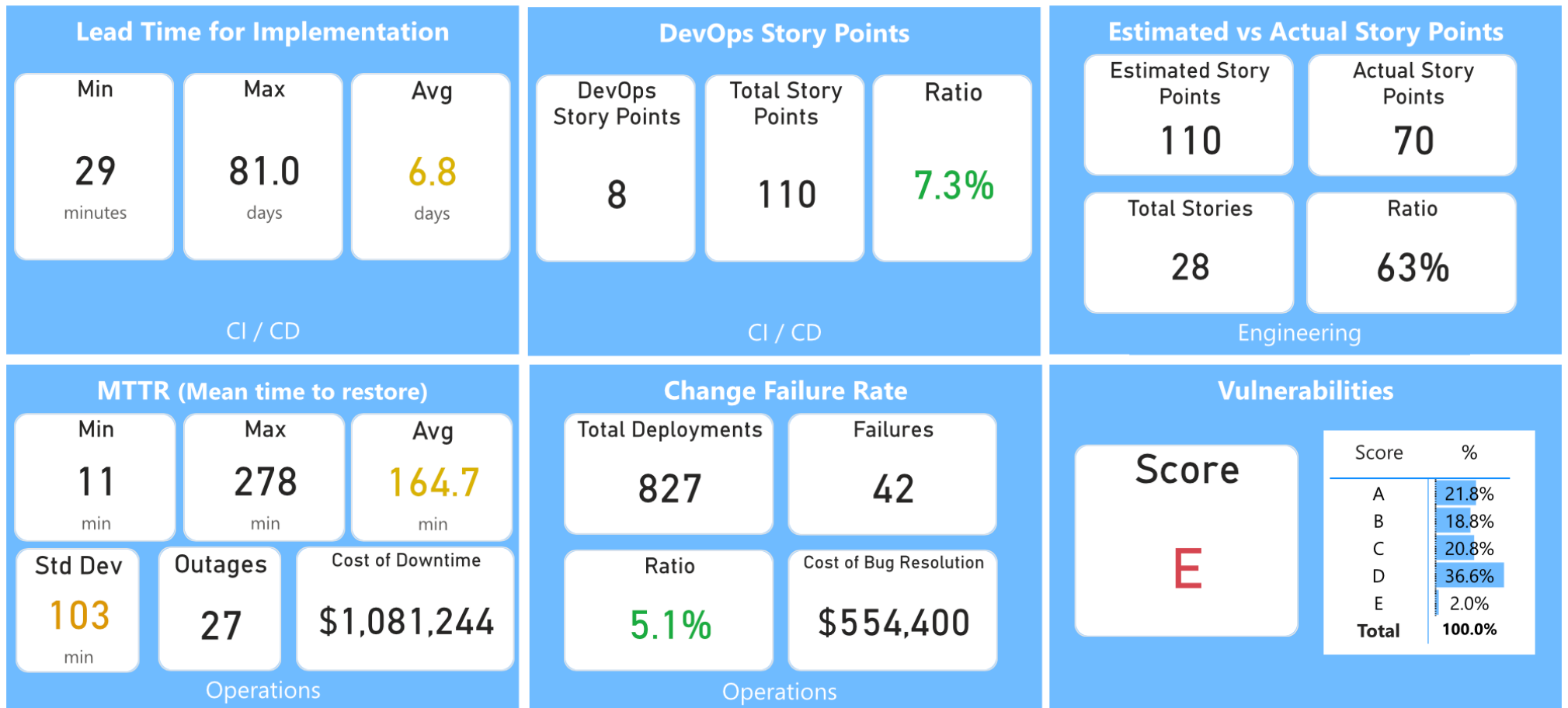
- Identificación de los objetivos de negocio
- Medición de la cultura
- Creación de un ambiente de intercambio
- Creación de un equipo DevOps
- Diseño de procesos DevOps



Metricas mas habituales de DevOps

KPI	Low	Medium	High	Elite
Deployment Frequency	More than once a month	Between once/month and once/week	On-demand(multiple deploys per day)	On-demand(multiple deploys per day)
Lead Time for Changes	More than 1month	Between 1 week and 1 month	Less than 1 day	Less than 1hour
Mean Time to Restore Service(MTTR)	Between 1 day and 1 week	Less than 1 day	Less than 1hour	Less than 1hour
Change Failure Rate	31-45%	16-30%	0-15%	0-15%
Speed of Deployment	More than 1 day	Less than 1 day	Less than 1hour	Less than 1hour
Test Code Coverage	20% or less	Between 20% and 80%	More than 80%	More than 80%
Security Vulnerability Score	Simply reporting all CVEs	No Critical CVEs in Prod	No Critical or High CVEs in Prod	Only Low CVEs in Prod

Mostrar el Valor de DevOps



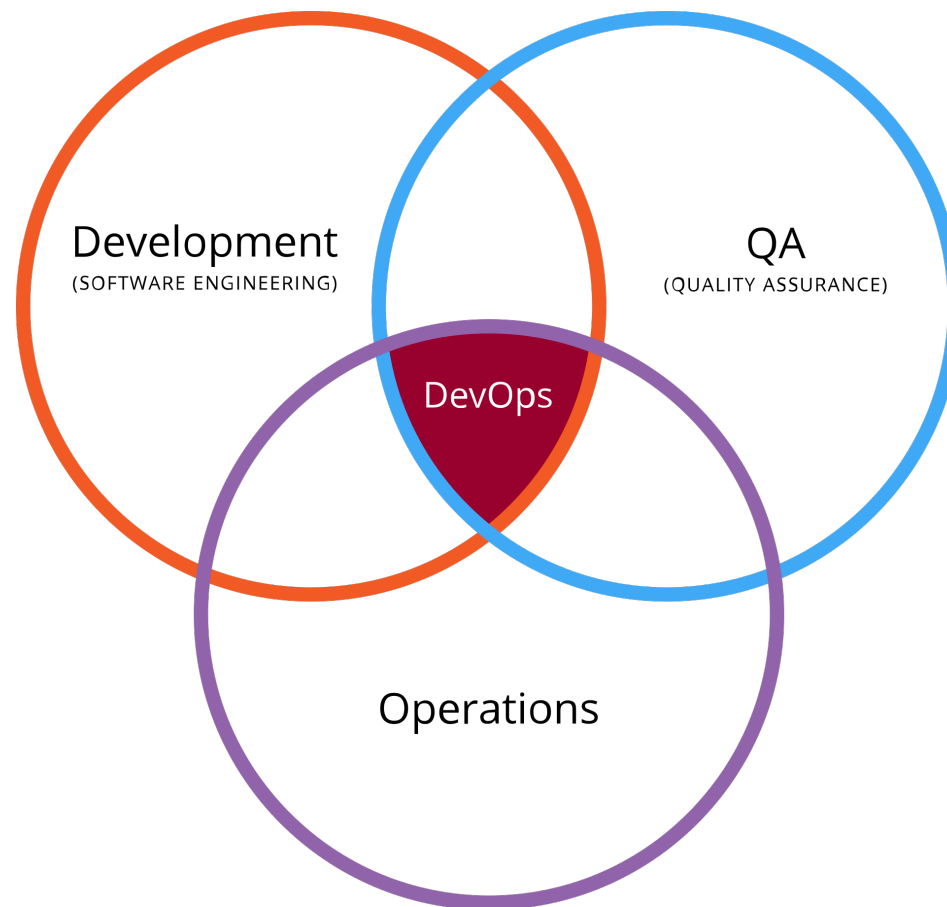
Medición de la cultura

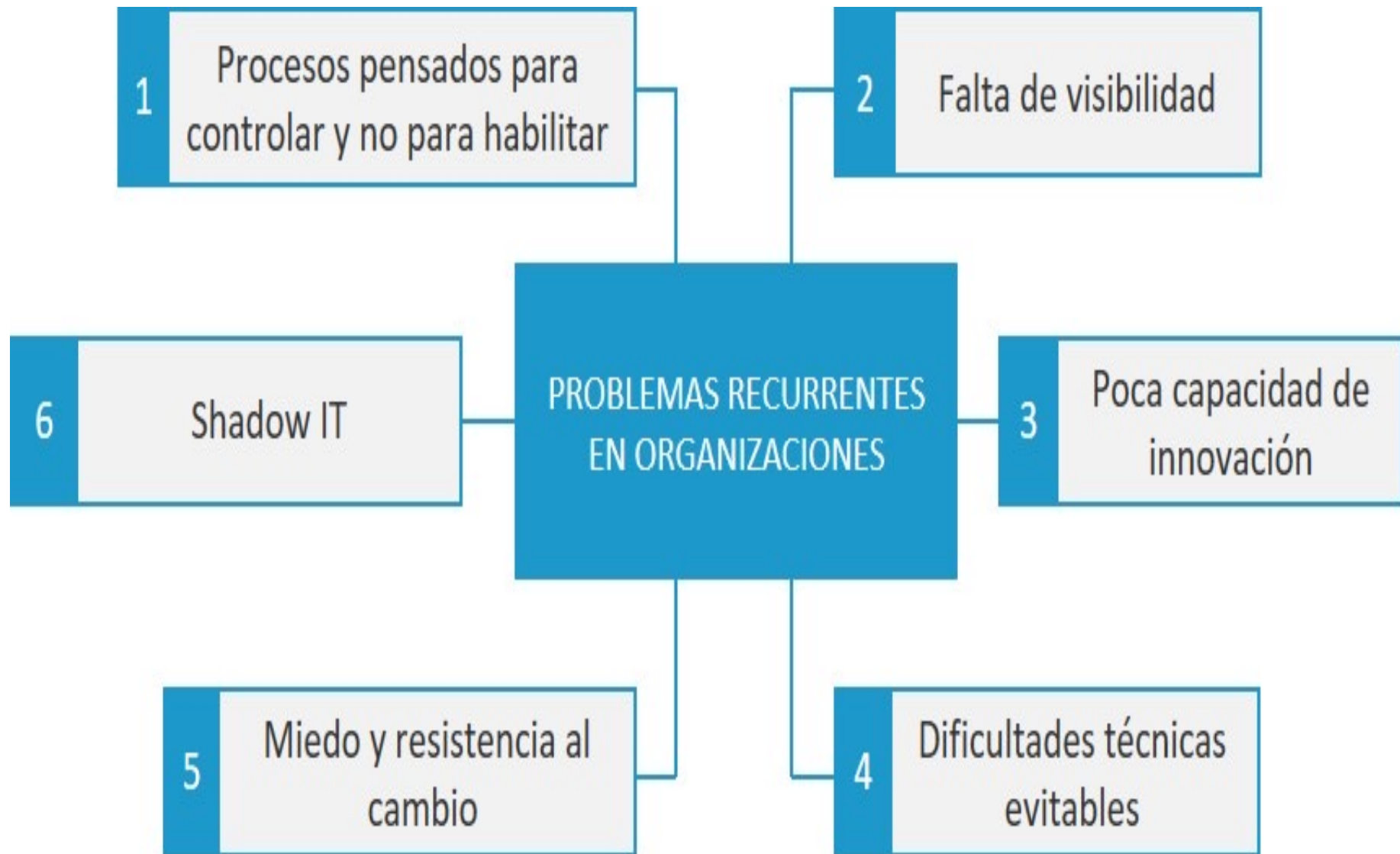
- Es importante conocer los retos y riesgos, y para ello podemos usar:
 - Sesiones de trabajo
 - Encuestas
 - Workshops
 - Otros

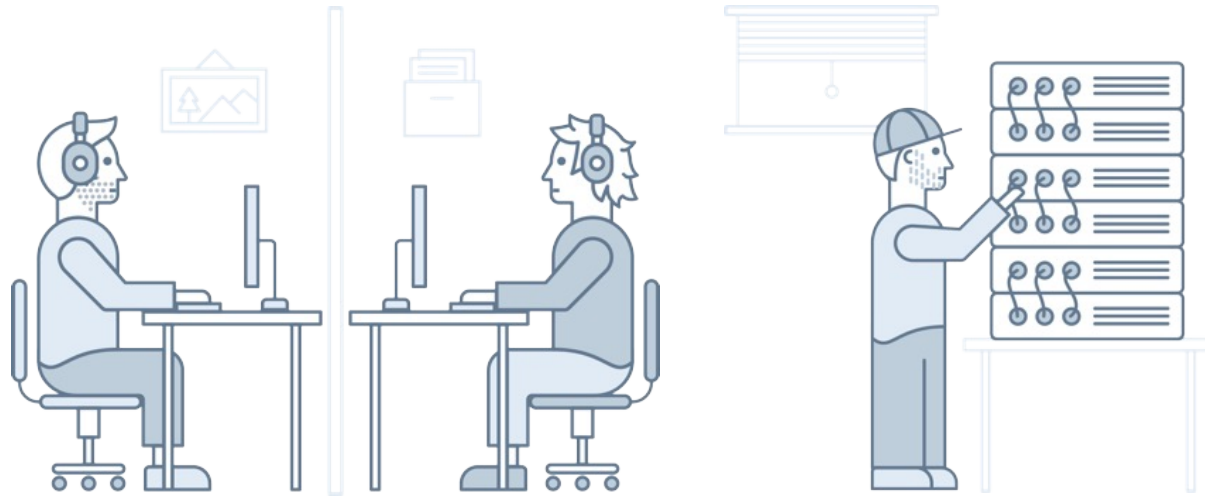


- <https://www.devops-research.com/quickcheck.html#questions>

Perfil de DevOps







Tema 3: DevOps Dojos.

DevOps dojos

```
graph TD; A[DevOps dojos] --> B[Dojos tecnológicos]; A --> C[Cómo crear un Dojo]; A --> D[Selección de nuevos equipos]; A --> E[Medir el éxito del Dojo];
```

Dojos tecnológicos

Los dojos pueden ser un vehículo singularmente poderoso para acelerar la transformación a través de estos cuatro pilares de cambio que ya mencionamos. Dojo significa «lugar del camino» en japonés. El Dojo, en nuestro contexto, es un espacio diseñado para albergar una experiencia de aprendizaje inversiva donde equipos completos vienen a aprender ingeniería moderna, productos y prácticas ágiles.

Cómo crear un Dojo

- ▶ Alinear a la dirección de la organización
- ▶ Encontrar un espacio apropiado
- ▶ Seleccionar a los entrenadores

Selección de nuevos equipos

Equipos colocados que puedan asistir al Dojo en persona, motivados, centrados en producto, multidisciplinares y con tiempo.

Medir el éxito del Dojo

No existe una única forma de medir, pero idealmente se deben usar métricas asociadas con el aumento en la eficiencia, la velocidad, la disminución y el aumento de errores en el valor que se entrega.

Dojos tecnológicos

Los Dojos pueden ser un vehículo singularmente poderoso para acelerar la transformación a través de estos cuatro pilares de cambio que ya mencionamos. «Dojo» significa ‘lugar del camino’ en japonés. El Dojo, en nuestro contexto, es un espacio diseñado para albergar una experiencia de aprendizaje inmersiva donde equipos completos vienen a aprender ingeniería moderna, productos y prácticas ágiles.



Mantra

Los Dojos optimizan la velocidad y la eficiencia en lugar de optimizar los costes. De ello se deduce que la producción rápida de valor para el negocio es lo que hace que compense la inversión.

Se compensa por que:



La mayor parte del esfuerzo desperdiciado en la entrega de TI proviene del hecho de que los equipos pasan tiempo construyendo cosas que no añaden valor.

Cómo crear un Dojo



Creación de un espacio apropiado

- Configuración del espacio:
 - Colocación de la mesas para facilitar la co
 - Pizarras y herramientas de visualización.
 - Espacios abiertos.
- Centro de comunicación y colaboración.
- Diseminadores de información (dashboards).
- Espacio que fomente la diversión y la creatividad



Equipo clave para el Dojo

- DevOps Coaches (Entrenador/Facilitador)
 - Agile Coach (Entrenador ágil)
 - Product Coach (Entrenador de producto)
 - Technology Coach (Entrenador de tecnología)
 - Coach Experto (opcional)
 - Gerente o director de operaciones del Dojo (opcional)
-
- **Coach Principal**
 - **Gerente**



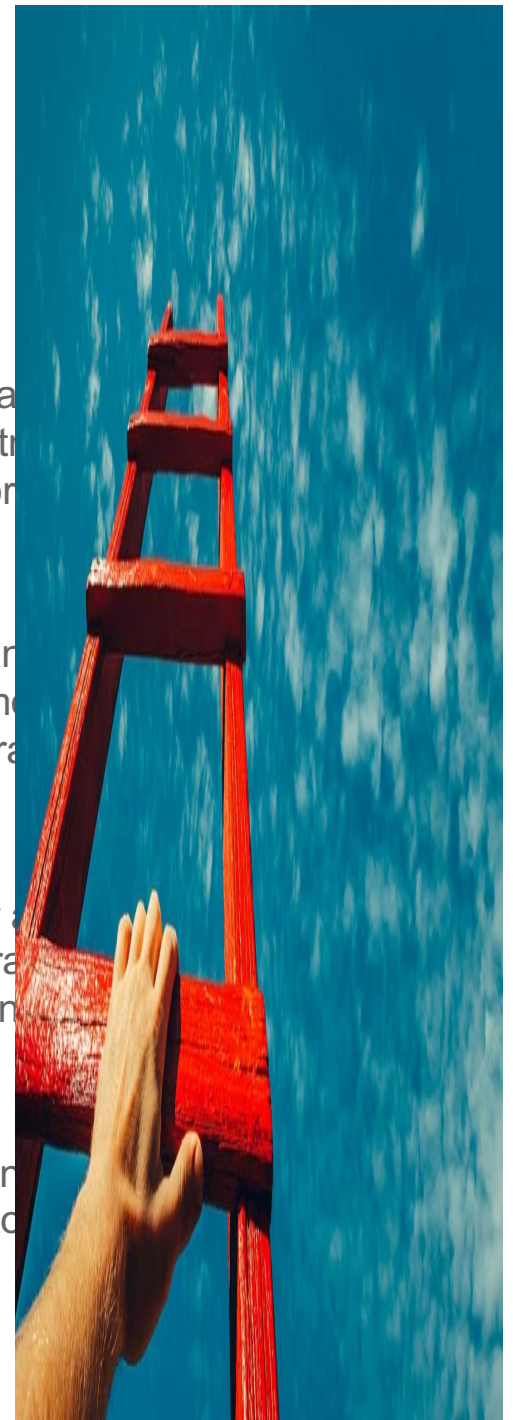
Selección de nuevos equipos

Mejores Candidatos:

- Un equipo multidisciplinar completo, un equipo de «dos pizzas» (ocho personas).
- Equipos que trabajan físicamente juntos (más adelante, en el tema, se analizan otras opciones para cuando esto no sea factible).
- Equipos que sean capaces de comprometerse, al menos, seis horas al día por un período de cuatro a ocho semanas consecutivas.
- Individuos que puedan comprometerse con el aprendizaje y que combinen la vocación Ágil con el foco en el producto y la tecnología.

Formato del Dojo

- **Desafío:** los equipos entran en el Dojo con un reto propio diseñado a medida. Estos Dojos deben incluir a todo el equipo: product owner, ingenieros, maestros diseñadores. El enfoque del desafío es una transformación holística de las prácticas de trabajo de un equipo.
- **FlashBuild:** este formato es más pequeño, suele durar menos de una semana para equipos que intentan resolver un problema técnico muy específico, cómo es un pipeline de CI/CD, aplicar un framework de desarrollo o de pruebas, configurar en nube o contenedores, etc.
- **Talleres:** se trata de eventos, de una o varias horas, diseñados para educar en temas específicos. Ejemplos de estos pueden ser eventos enfocados en migrar infraestructura a nube de forma eficiente, implantar Kubernetes, securizar el código, implementar pruebas basado en pruebas, etc.
- **Talleres de líderes:** es un taller de inmersión que no suele durar más que un día, combina experiencias prácticas junto con una conferencia sobre los conceptos fundamentales que los equipos están adoptando.



Ofertas Basicas / Menu

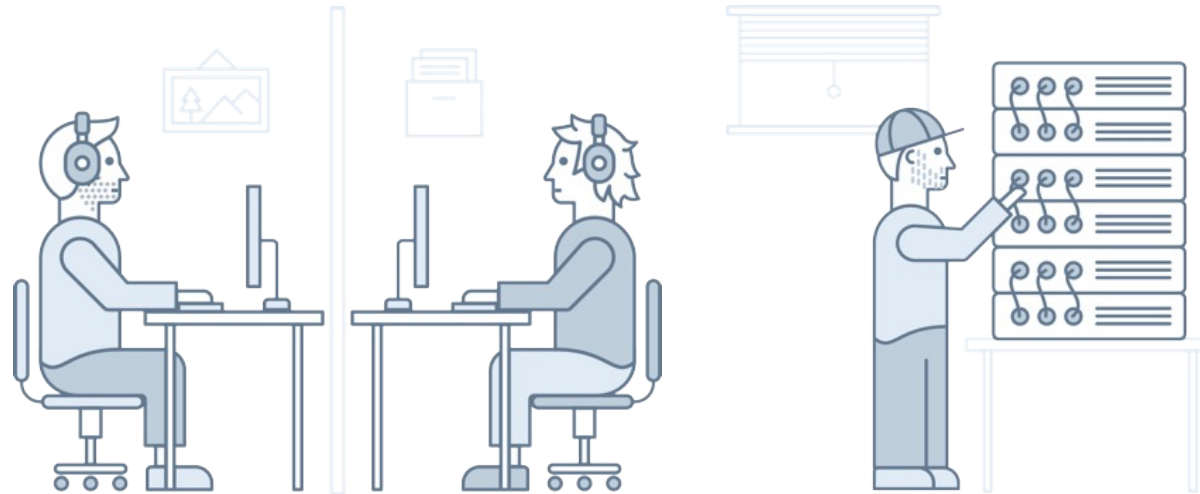
- Normalmente es una lista paquetizada de formacion, workshops, actividades, recursos, ayudas que los equipos pueden consumir idealmente en un modelo “self service”... o mediante un sistema de planifciacion sencillo.



Variaciones del Dojo Standard

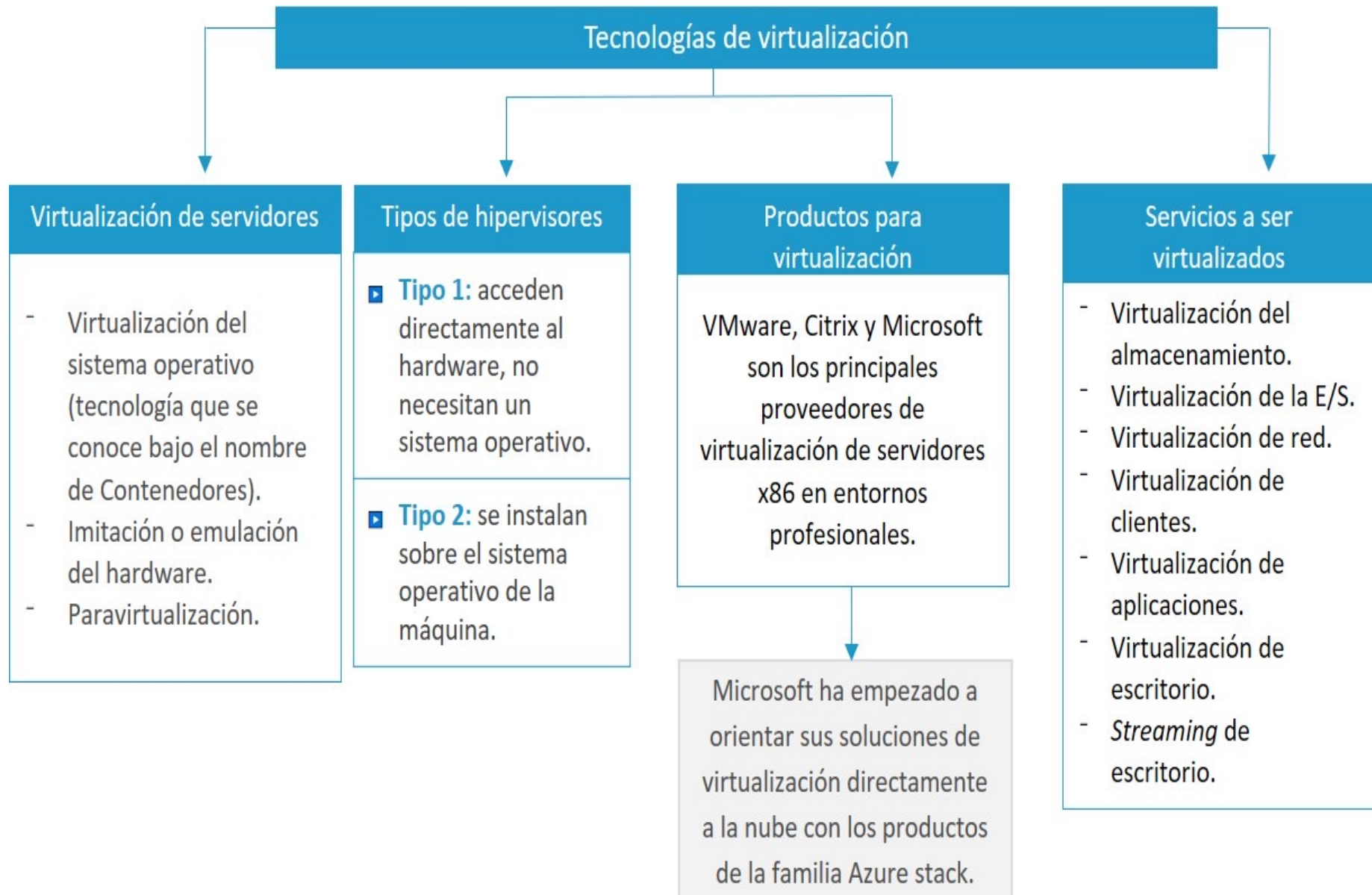
- Equipos distribuidos o remotos
- Falta de espacio destinado a la formación
- Dojo in a box
- Dos por cuatro





Tema 4:

Tecnologías de Virtualización.

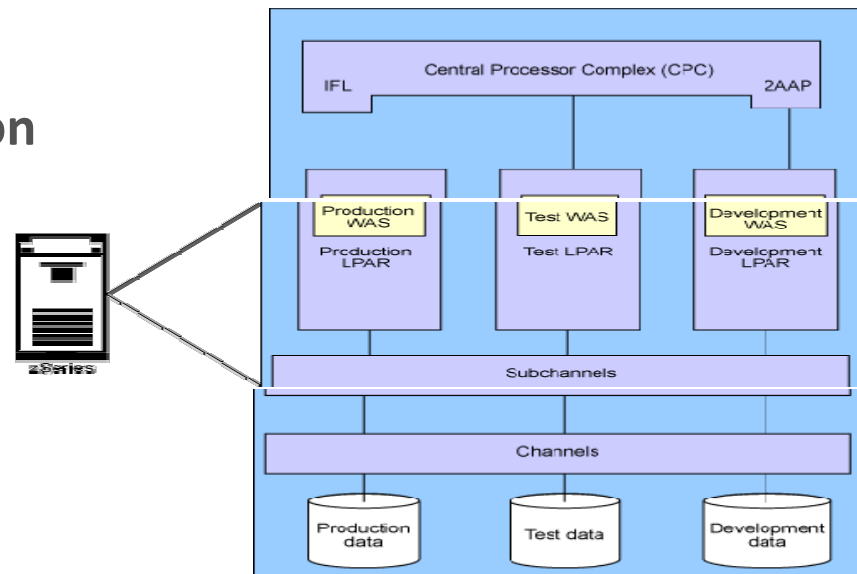


Historia de la virtualización

La virtualización empezó a desarrollarse en la década de 1960 para **particionar los mainframes de gran tamaño** a fin de mejorar su utilización

VMware reinventó la virtualización para la plataforma x86 en la década de 1990 para abordar los problemas de infrautilización y de otra índole, a lo largo de un proceso que obligó a superar gran cantidad de desafíos.

Mainframe virtualization



Desencadenantes

Problemas:

- Baja utilización de la infraestructura.
- Incremento de los costes de infraestructura física
- Incremento de los costes de gestión de TI
- Insuficiente protección ante desastres y failover
- Escritorios de usuario final de mantenimiento elevado



Monitor de máquina virtual o “hipervisor”

Un **hipervisor**, también llamado monitor de máquina virtual (*virtual machine monitor*, VMM), es una plataforma de virtualización que permite utilizar, a la vez, múltiples sistemas operativos en un equipo. Los hipervisores se pueden clasificar en dos tipos:

Tipo 1 (nativo, *baremetal* o *unhosted*)

Tipo 2 (*hosted*)

Tipo 1 (nativo, baremetal o unhosted)

Software que se ejecuta directamente sobre el hardware real del equipo para controlar el hardware y monitorizar los sistemas operativos virtualizados.

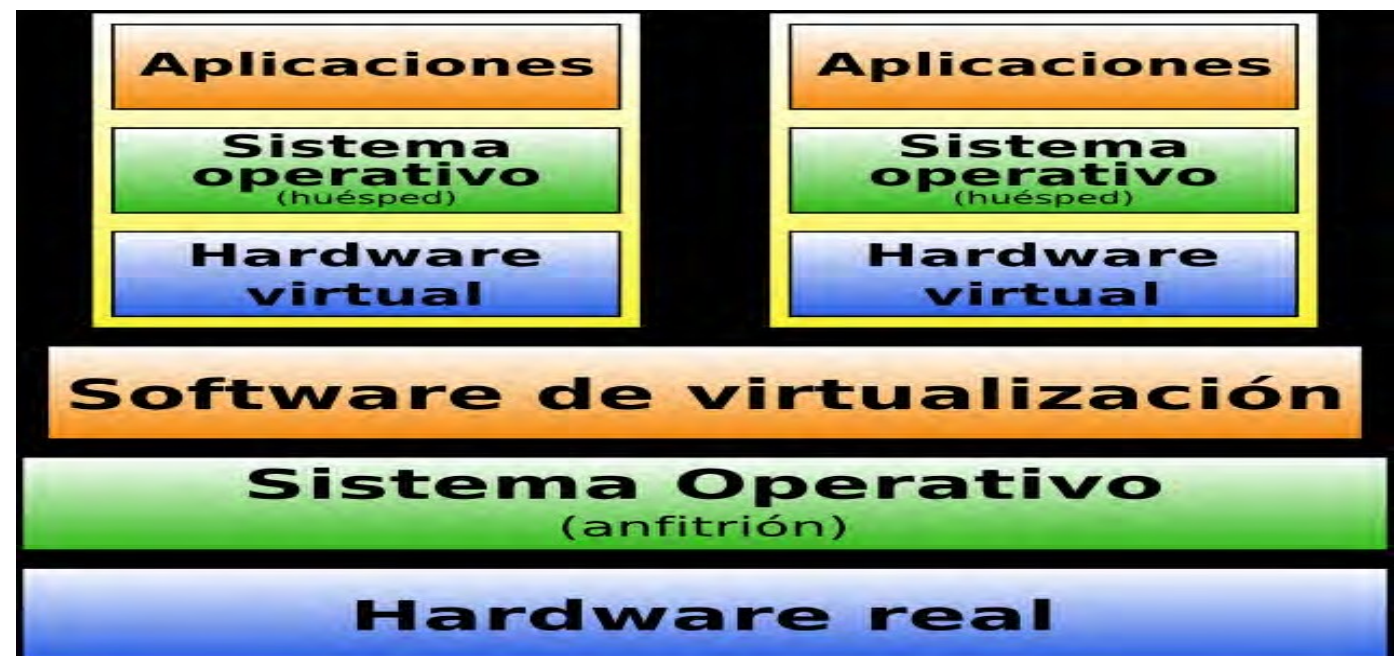
Los sistemas virtualizados se ejecutan en otro nivel por encima del hipervisor.



Tipo 2 (hosted)

Aplicación que se ejecuta sobre un sistema operativo convencional (Linux, Windows, MacOS) para virtualizar sistemas.

De esta forma la virtualización se produce en una capa más alejada del hardware si lo comparamos con los hipervisores de tipo 1. Lógicamente esto hace que el rendimiento sea menor en los hipervisores de tipo 2.



Las cinco razones para virtualizar:

- **Sacar más provecho de los recursos actuales:** se pueden agrupar los recursos de infraestructura comunes y dejar atrás el modelo heredado de “una aplicación por servidor” – a eso se le llama una consolidación de servidores.

Ventajas:

- Reducción en los costes operativos y de hardware en mas del 50%, y los costes energéticos en un 80%, por cada carga de servidor virtualizada.
- Reducción en el tiempo necesario para el aprovisionamiento de nuevos servidores hasta un 70%.
- Disminución en la cantidad de interrupciones del servicio y mejore la fiabilidad

Las cinco razones para virtualizar:

- Reducción de los costes consolidando el hardware y aumentando el uso de los servidores:
 - Aumentar la utilización del hardware existente del 5 o el 15% hasta un 80%.
 - Reducir los requisitos de hardware en una relación de 10:1 o más.
- Gestión de infraestructura virtual desde un solo punto de control
 - Acelerar el tiempo de aprovisionamiento entre un 50 y un 70%.
 - Gestionar máquinas virtuales desde una ubicación central.
 - Supervisar el rendimiento de las máquinas virtuales y sus hosts.

Las cinco razones para virtualizar:

2. Disminución en los costes del centro de datos reduciendo la infraestructura física y aumentando la proporción de servidores por administrador:

una menor cantidad de servidores y de hardware de TI se traduce en menos requisitos de espacio físico, así como menos consumo energético y de refrigeración.

Unas herramientas de gestión adecuadas permiten aumentar la proporción de servidores por administrador, de modo que también son menos los requisitos de personal.

Las cinco razones para virtualizar:

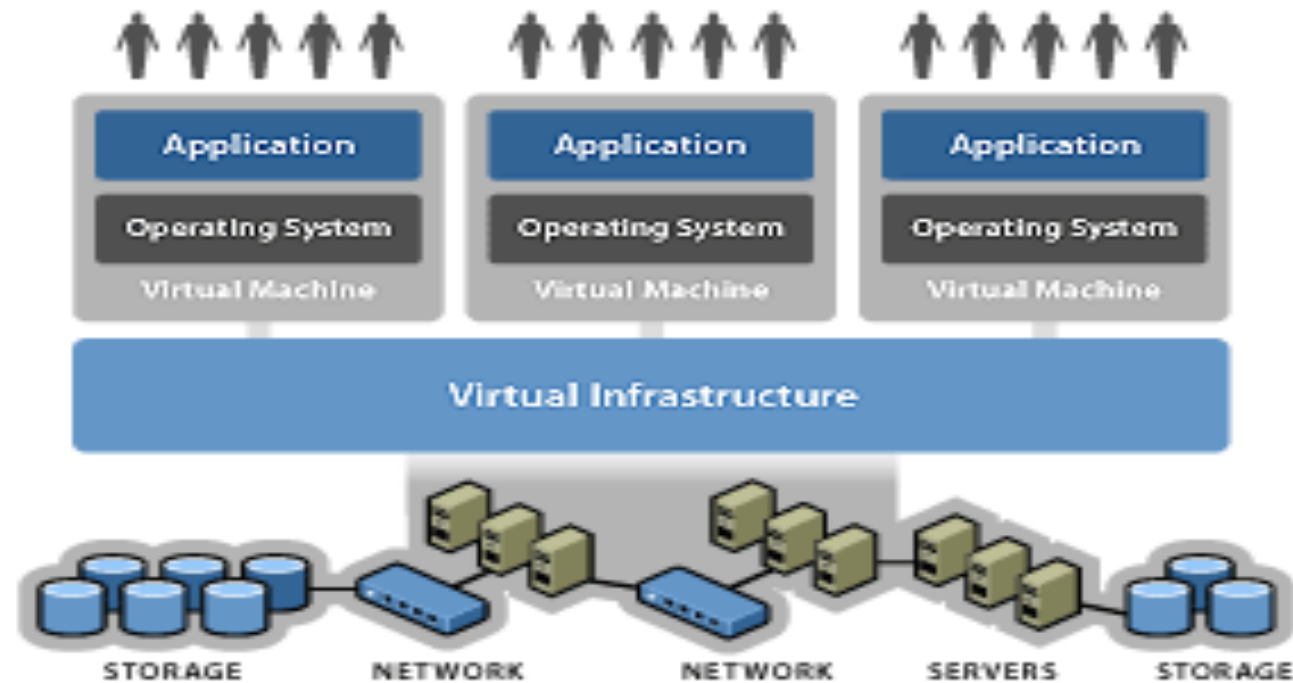
3. **Aumentar la disponibilidad del hardware y las aplicaciones para mejorar la continuidad del negocio:** Realización backup con mas seguridad y migración de entornos virtuales completos sin interrupción alguna del servicio.
Eliminación de las interrupciones del servicio planificadas y recuperación al instante de los incidentes imprevistos.
4. **Flexibilidad operativa:** responder a los cambios del mercado con una gestión dinámica de los recursos, con un aprovisionamiento de servidores acelerado y con una mejora de la implementación de escritorios y aplicaciones.

Las cinco razones para virtualizar:

5. Mejorar la capacidad de gestión y seguridad de los escritorios:

implementación, gestión y supervisión de entornos de escritorios seguros a los que los usuarios pueden acceder de forma local o remota, con conexión de red, desde casi cualquier ordenador de escritorio, portátil o tablet PC.

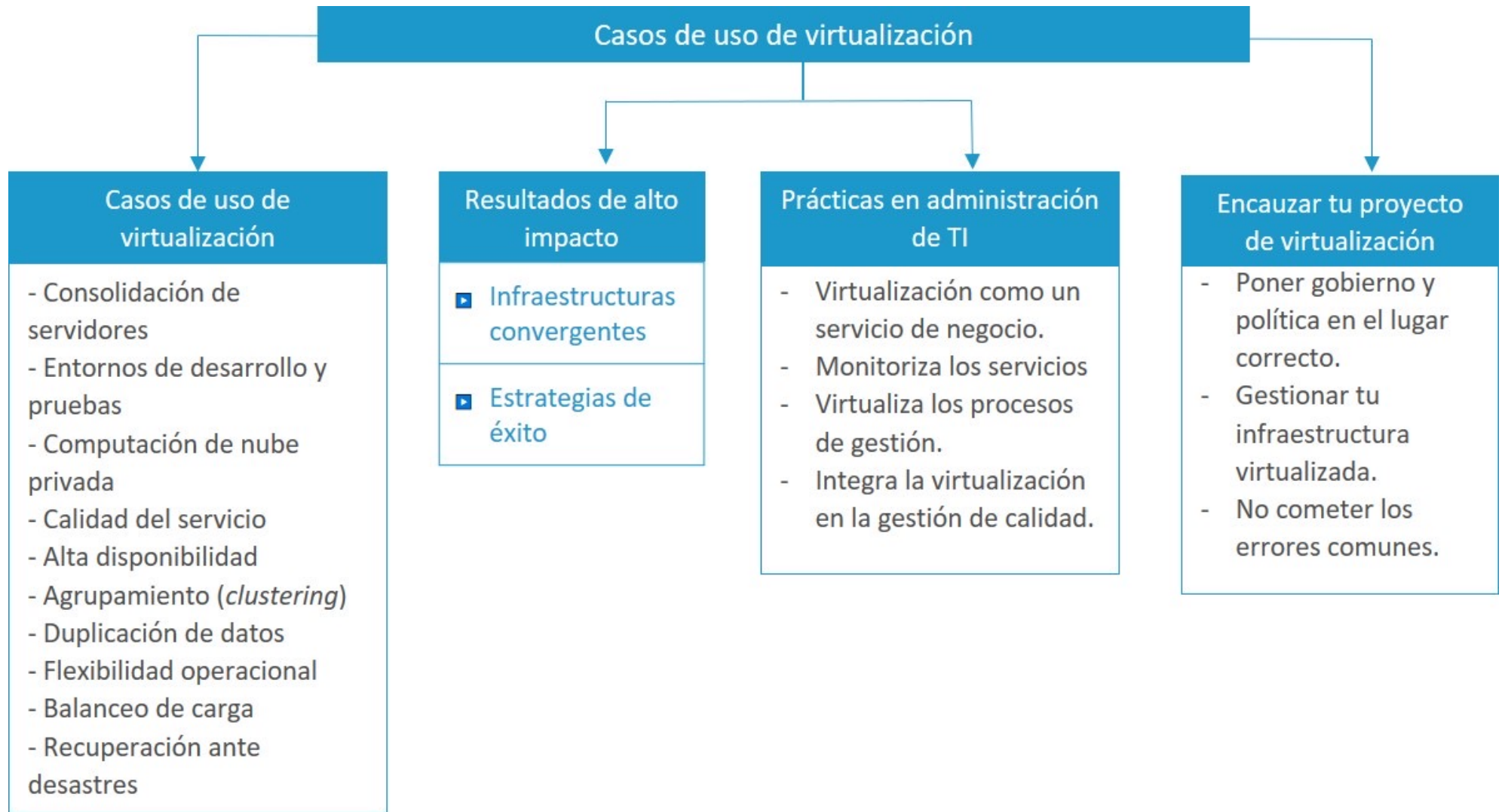
Infraestructura virtual





Tema 5:

Casos de Uso de Virtualización

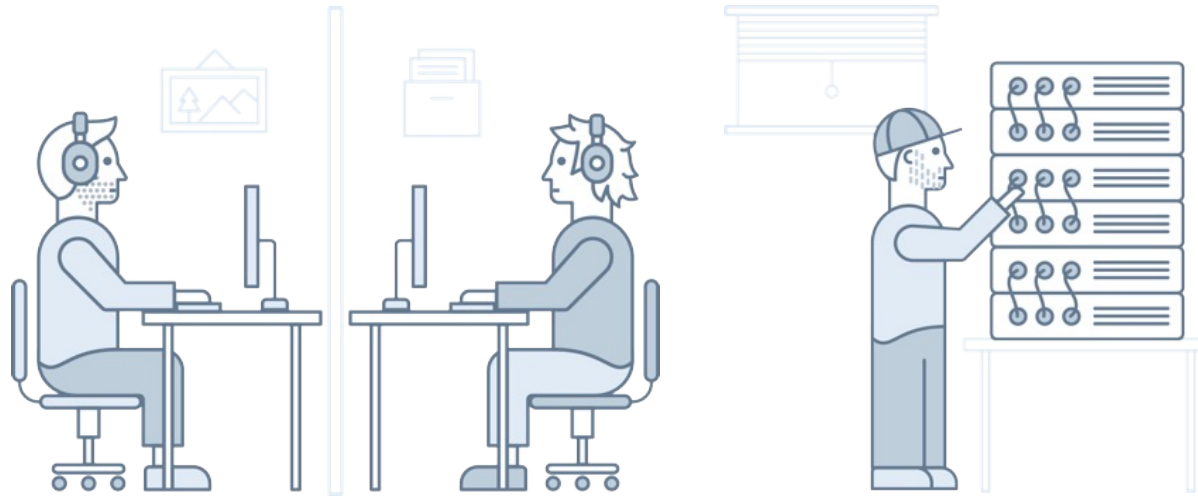


SECTOR Y OBJETIVOS	QUIEN	QUE	VENTAJAS
Educación - VDI		Recursos Computacionales a estudiantes	Disponibilidad y Eficiencia
Educación - Migración		Abandonar AIX hacia plataformas sostenibles	Reducción de Costes y Mantenimiento
Green IT	VARIOS 	Responsabilidad para el medio ambiente	Reducción de costes y mejor imagen
Salud - Homogeneidad		Simplificar infraestructuras	Menor Gestión
Hosting - Servicios 3ros		Cambio de negocio hacia el hosting virtual	Reducción de costes
Consolidación		Reducción de Legacy y homogenización	Reducción de costes
Continuidad		Garantizar la continuidad de servicio	Mantenimiento del negocio
Gestión - VDI		Teletrabajo y virtualización de escritorio	Disponibilidad y Eficiencia
Migración y automatización de pruebas		Eliminar legacy y automatizar test	Reducción de defectos y eficiencias

ElasticBox, Dena y Netflix

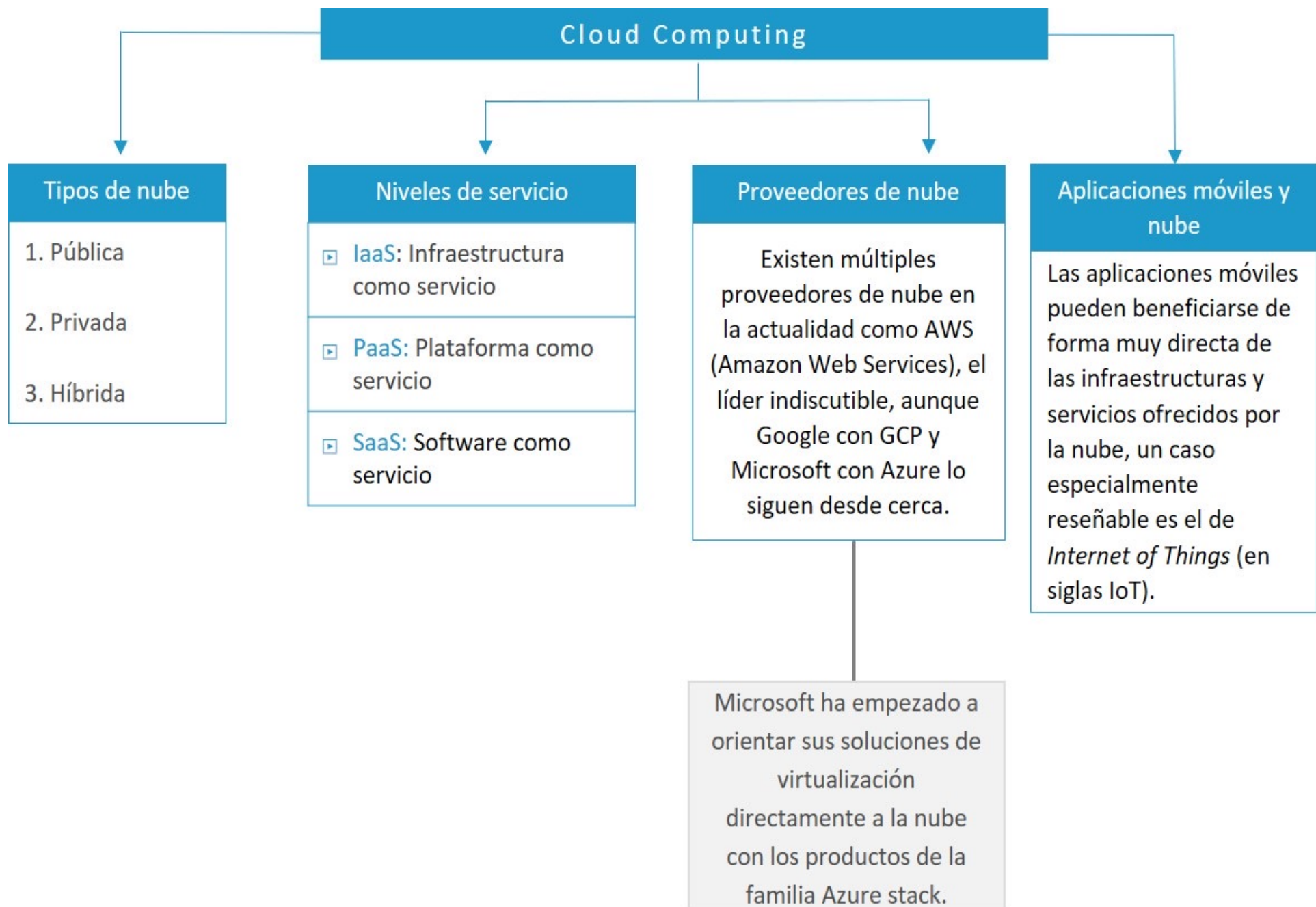


DeNA



Tema 6: Cloud Computing

Cloud Computing, DevOps y Cultura DevOps.



Cambio generacional

TECHNOLOGY

ECONOMIC

BUSINESS



Grandes ordenadores con terminales “tontos”

Optimizar para reducir costes.

Coste inicial muy alto.



PCs y servidores con servicios distribuidos

Optimizar la en la solución mas rápida.

Costes iniciales moderados caracterizados por licencias perpetuas.



Centros de datos con hardware de usar y tirar

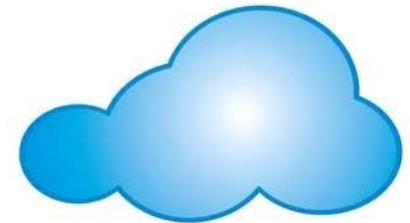
Optimizado para proporcionar elasticidad.

Pay as you go.

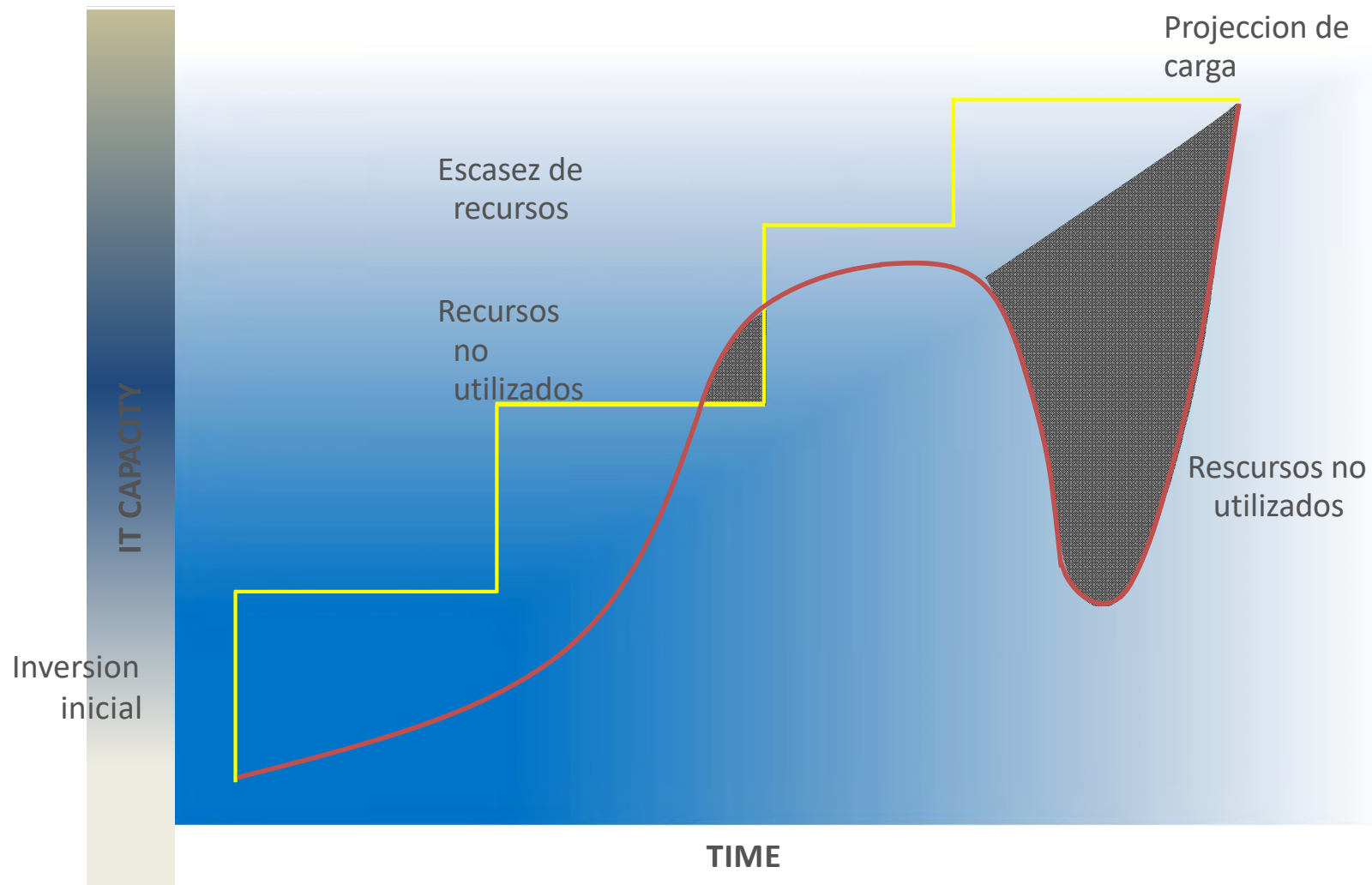
¿Qué es Cloud Computing, o Computación en Nube?

Dos definiciones de Cloud Computing bastante aceptadas son las siguientes:

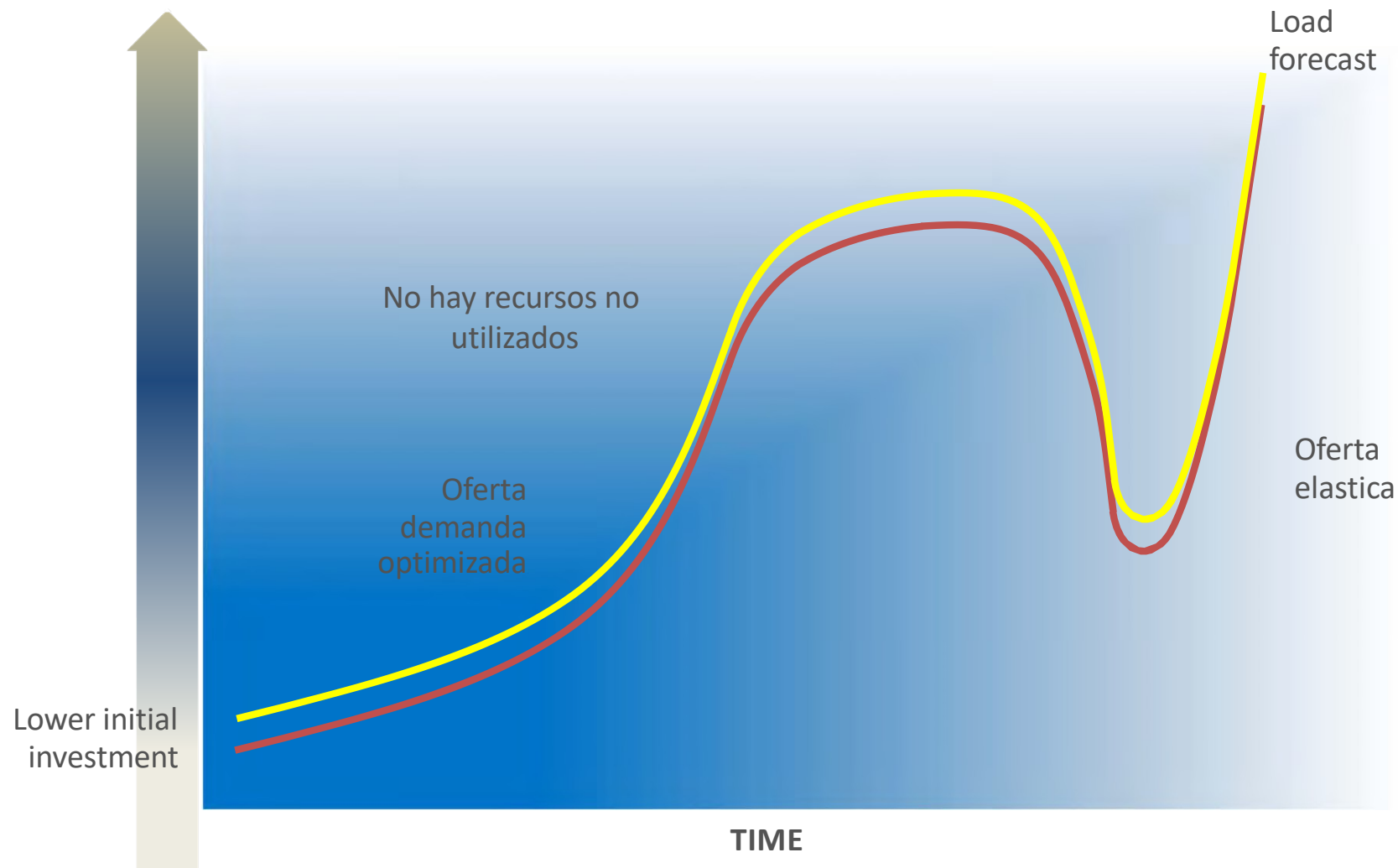
- “Un estilo de cómputo donde las capacidades T.I. en escala masiva son entregadas “como un servicio” a clientes externos usando tecnologías de Internet” **Gartner**
- “Es simplemente una metáfora de Internet: el creciente movimiento recursos de cómputo dentro de la red” **Sun Microsystems**



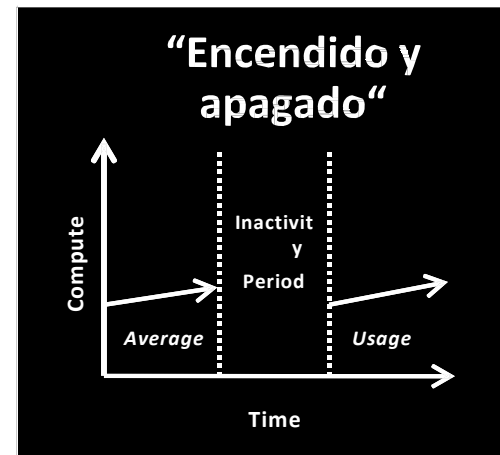
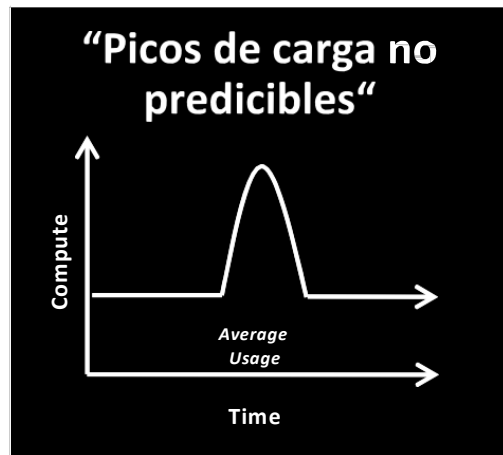
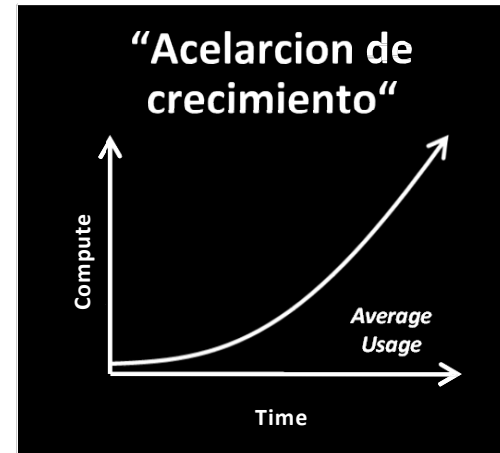
Distribución de Recursos: Modelo tradicional



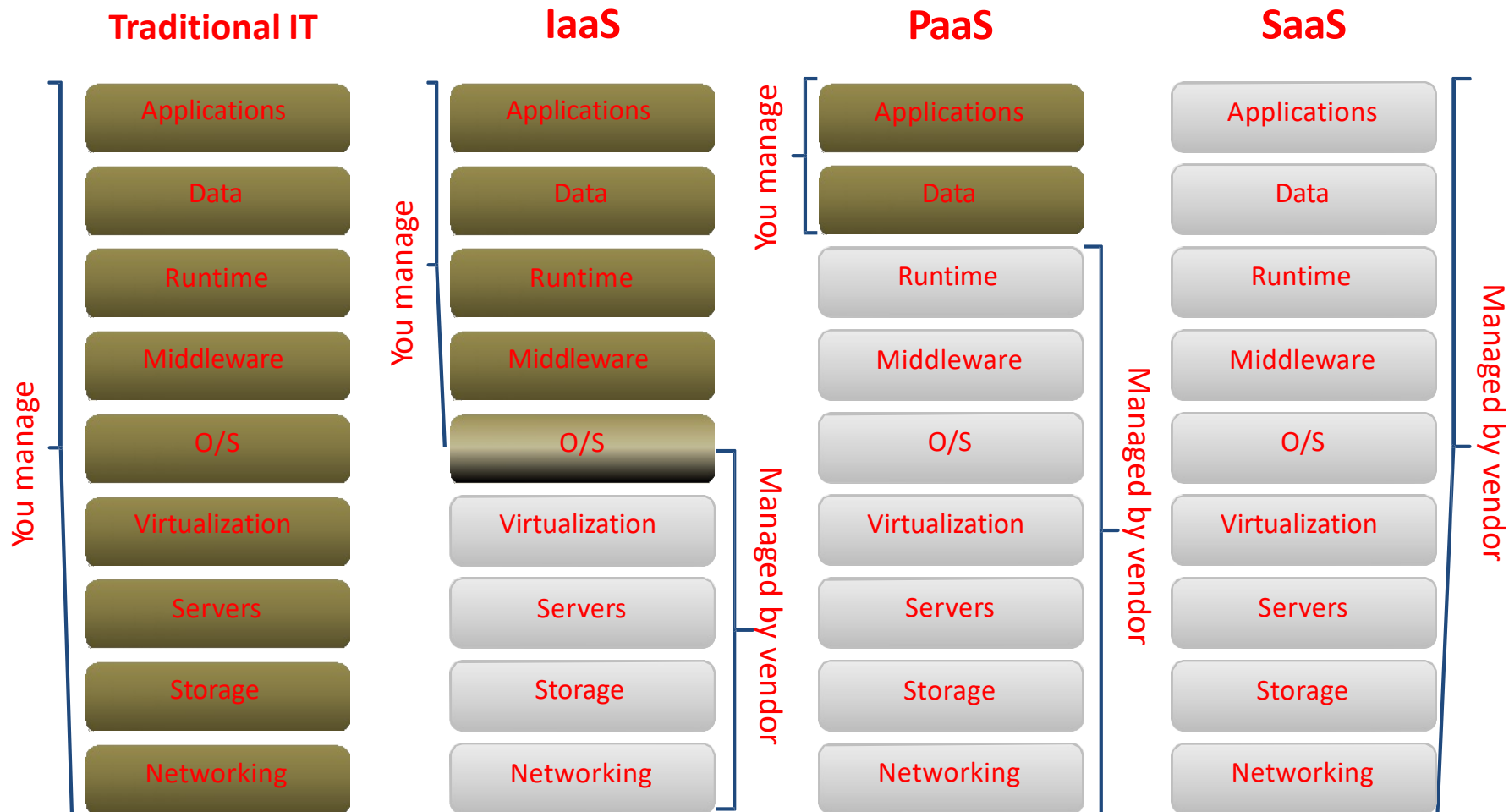
Distribución de Recursos: Modelo de nube



Tipos de elasticidad en la nube



Tipos de Cloud computing



Características



CLOUD PRIVADO

- Mejora la operación IT interna
- Mejoras modestas en costes
- Escalabilidad y flexibilidad limitadas
- Riesgo de obsolescencia
- Gestión limitada de picos de demanda.
- Proyectos a medida (Cloud Expertise)



CLOUD HÍBRIDO

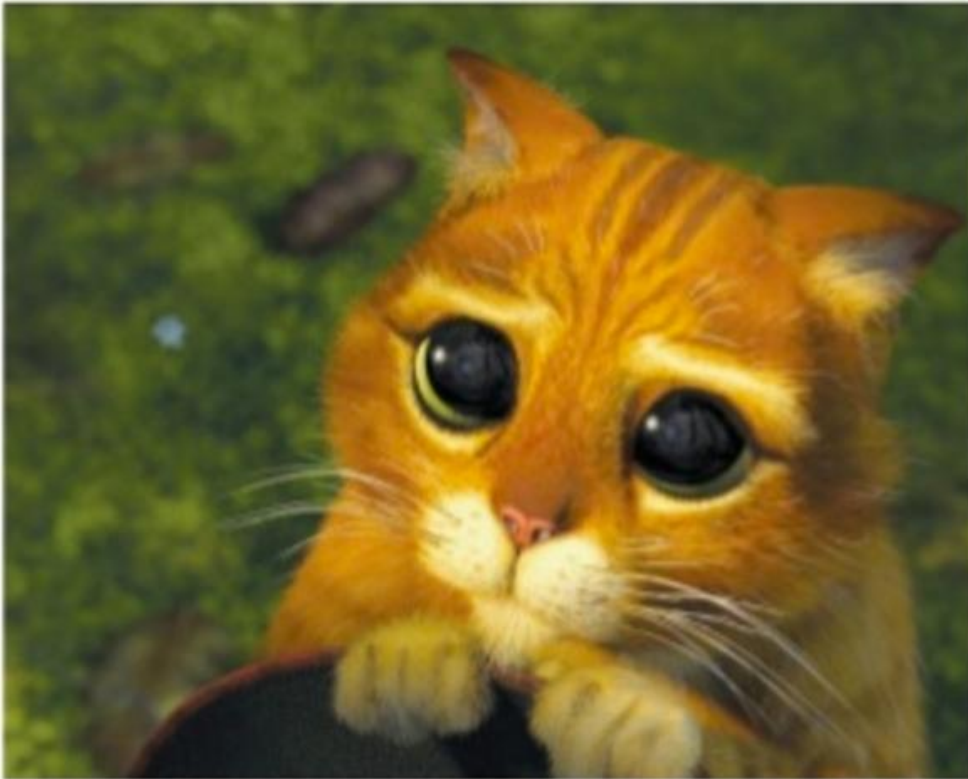
- Mix de los anteriores
- Incertidumbres sobre calidad
- Incertidumbres sobre seguridad
- Adecuado en escenarios de desbordamiento y poco críticos.
- Proyectos a medida (Cloud Expertise)



CLOUD PÚBLICO

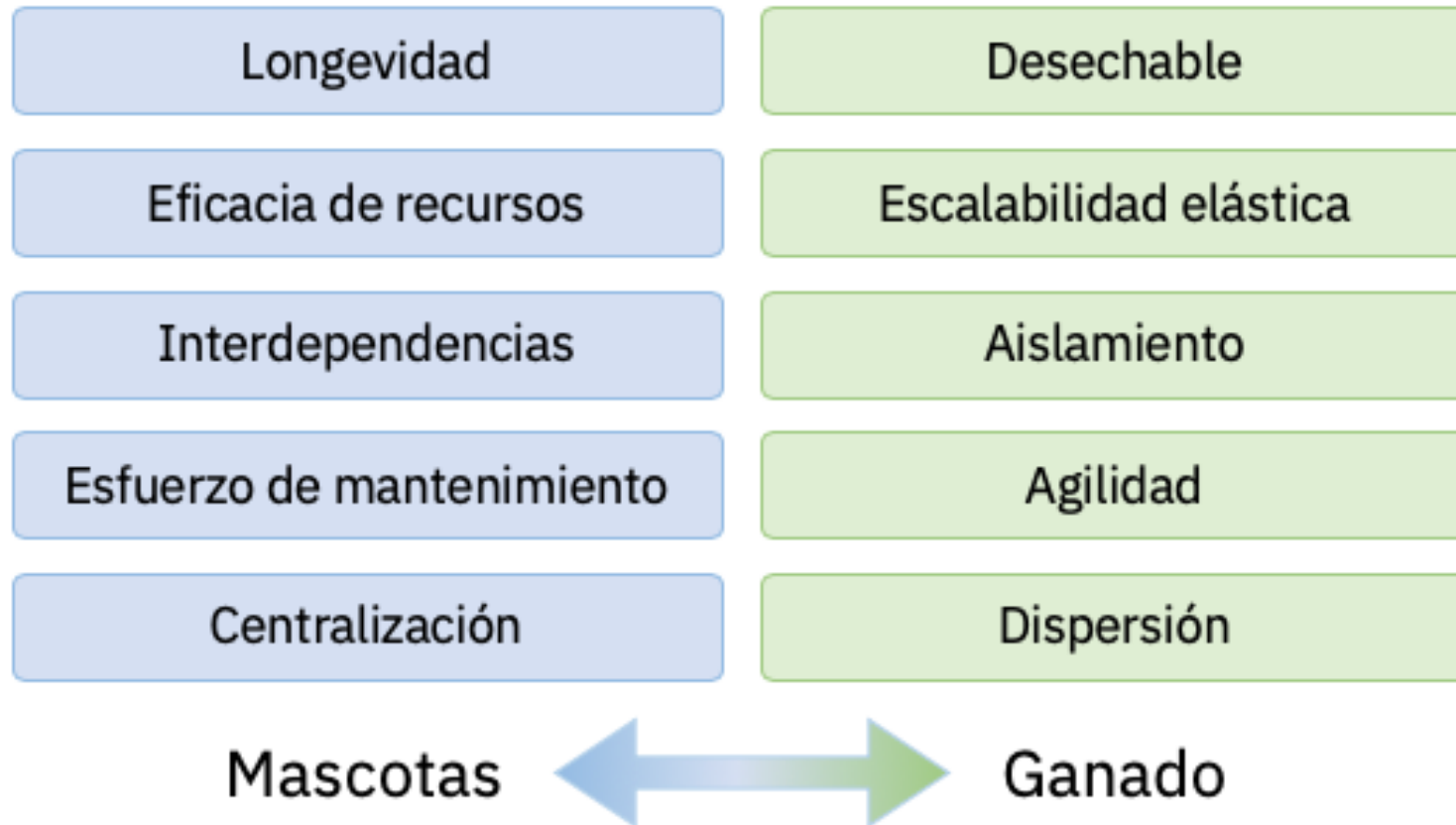
- Mayor escala, costes más bajos
- Dificil integración con 'Legacy'
- Incertidumbres sobre calidad y seguridad
- Mayor nivel de autogestión
- Modular y escalable

Cattle vs Pets (Granjas vs Mascotas)



Deferencias

medium.com/@nachoad





Tema 7: AWS

Amazon Web Services

Compute

-  **EC2**
Virtual Servers in the Cloud
-  **Lambda** PREVIEW
Run Code in Response to Events

Storage & Content Delivery

-  **S3**
Scalable Storage in the Cloud
-  **Storage Gateway**
Integrates On-Premises IT Environments with Cloud Storage
-  **Glacier**
Archive Storage in the Cloud
-  **CloudFront**
Global Content Delivery Network

Database

-  **RDS**
MySQL, Postgres, Oracle, SQL Server, and Amazon Aurora
-  **DynamoDB**
Predictable and Scalable NoSQL Data Store
-  **ElastiCache**
In-Memory Cache
-  **Redshift**
Managed Petabyte-Scale Data Warehouse Service

Networking

-  **VPC**
Isolated Cloud Resources
-  **Direct Connect**
Dedicated Network Connection to AWS
-  **Route 53**
Scalable DNS and Domain Name Registration

Administration & Security

-  **Directory Service**
Managed Directories in the Cloud
-  **Identity & Access Management**
Access Control and Key Management
-  **Trusted Advisor**
AWS Cloud Optimization Expert
-  **CloudTrail**
User Activity and Change Tracking
-  **Config** PREVIEW
Resource Configurations and Inventory
-  **CloudWatch**
Resource and Application Monitoring

Deployment & Management

-  **Elastic Beanstalk**
AWS Application Container
-  **OpsWorks**
DevOps Application Management Service
-  **CloudFormation**
Templated AWS Resource Creation
-  **CodeDeploy**
Automated Deployments

Analytics

-  **EMR**
Managed Hadoop Framework
-  **Kinesis**
Real-time Processing of Streaming Big Data
-  **Data Pipeline**
Orchestration for Data-Driven Workflows

Application Services

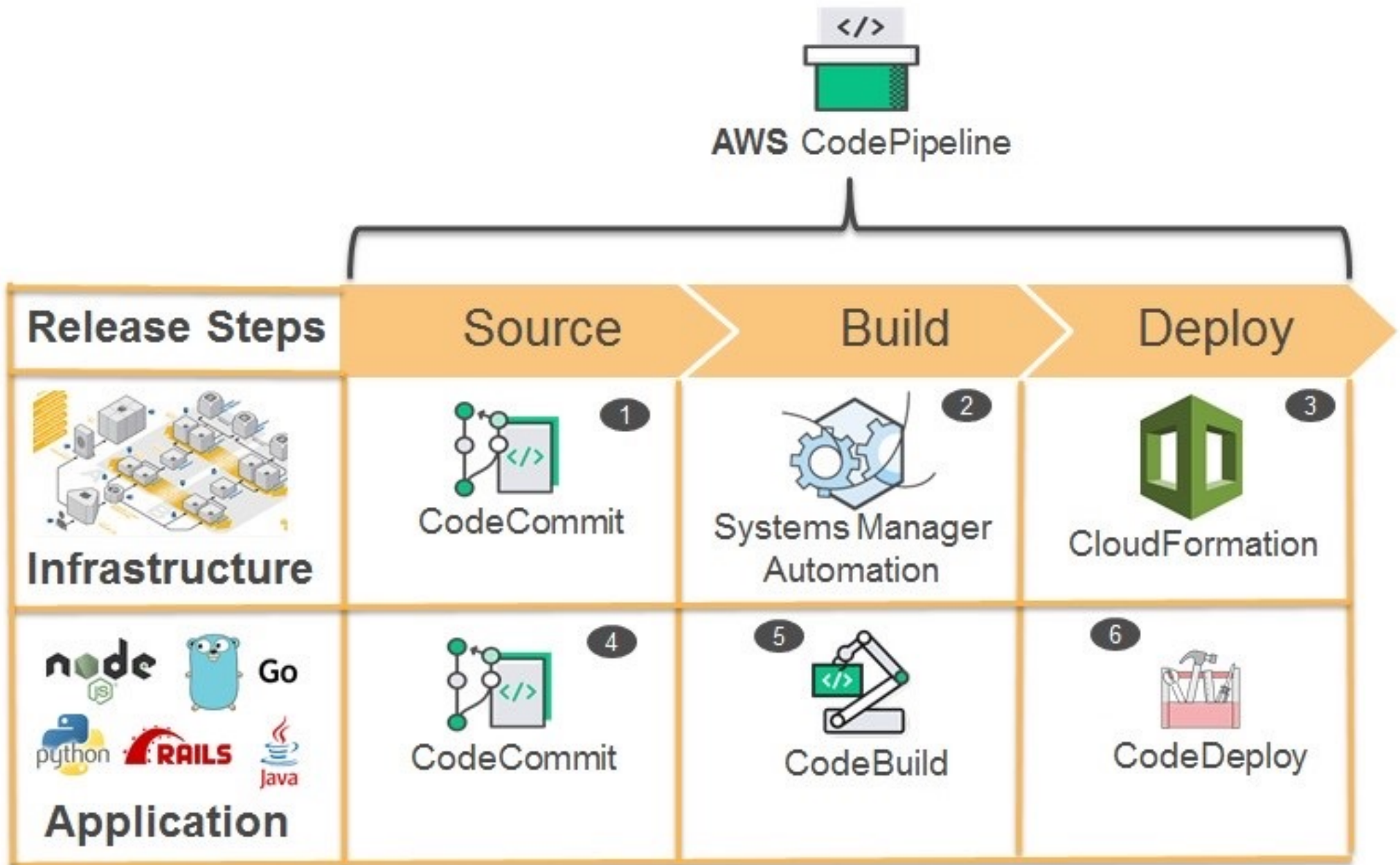
-  **SQS**
Message Queue Service
-  **SWF**
Workflow Service for Coordinating Application Components
-  **AppStream**
Low Latency Application Streaming
-  **Elastic Transcoder**
Easy-to-use Scalable Media Transcoding
-  **SES**
Email Sending Service
-  **CloudSearch**
Managed Search Service

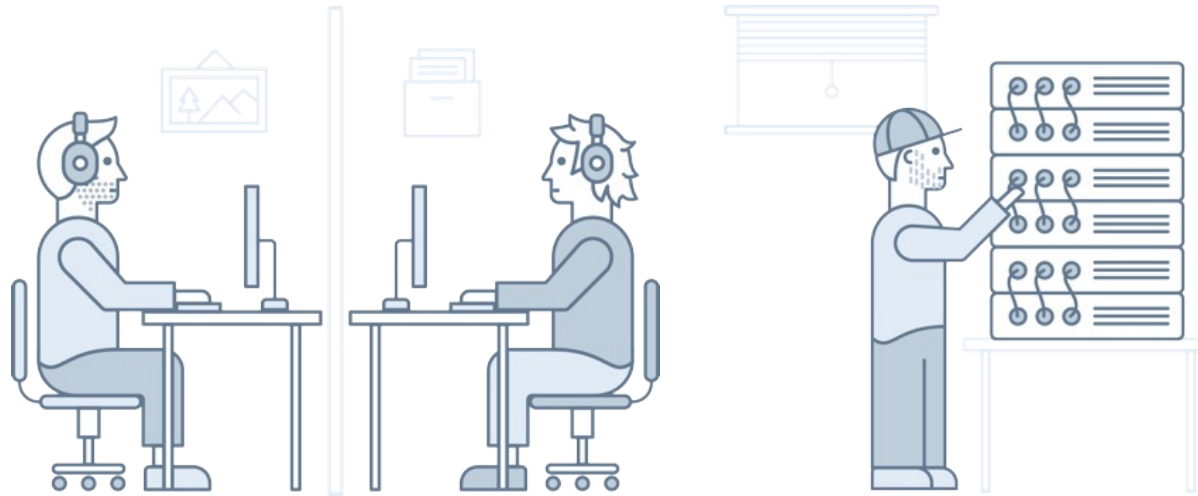
Mobile Services

-  **Cognito**
User Identity and App Data Synchronization
-  **Mobile Analytics**
Understand App Usage Data at Scale
-  **SNS**
Push Notification Service

Enterprise Applications

-  **WorkSpaces**
Desktops in the Cloud
-  **Zocalo**
Secure Enterprise Storage and Sharing Service





Tema 8: Infraestructura como código.

INFRAESTRUCTURA COMO CÓDIGO (IaC)

PRINCIPIOS de IaC

Automatiza y optimiza el ciclo de vida del desarrollo

Reutilización del código implementado en una infraestructura en otras

Control de versiones como cualquier software

Minimiza costes y esfuerzo

CICLO DE VIDA DE LOS RECURSOS DE INFRAESTRUCTURAS

Aprovisionamiento de recursos

Gestión de configuración

Supervisión y rendimiento

Gobierno y conformidad

Optimización de recursos

HERRAMIENTAS DE IaC

AWS CloudFormation

Terraform

Google Cloud
Deployment Manager

Chef Infra

Red Hat Ansible
Tower

IaC en AWS - AWS CLOUDFORMATION

PLANTILLAS

STACKS / PILAS

FLUJO DE TRABAJO

1. Crear una plantilla de AWS CloudFormation.
2. Salvar la plantilla localmente o en un cubo S3.
3. Crear una pila de AWS CloudFormation referenciando la plantilla.
4. Ejecutamos.

¿A qué llamamos IaC?

La infraestructura como código (IaC) es un enfoque para la gestión de infraestructuras de sistemas de TI que se basa en el uso de archivos de configuración repetibles para generar entornos de implementación consistentes para el desarrollo de CI/CD.

La infraestructura como código (en siglas, IaC) trata la infraestructura de sistemas como un software.

y... en consecuencia adquiriendo muchas de las ventajas del Desarrollo de Software, incluyendo por supuesto:

- Automatización.
- Repetibilidad.
- Predictibilidad.
- Trazabilidad.
- Versionado.
- Etc.

¿A qué llamamos IaC?

IaC Vs Op. Manual

IaC		Op. Manual	
1	Alta eficiencia: automatiza la mayor parte de la administración de los recursos, que conlleva a optimizar el ciclo de vida del desarrollo del software.	1	Costos altos: la necesidad de capital humano puede ir hacia otras necesidades del negocio más importantes.
2	Reutilización: una vez se ha escrito el código para una infraestructura, este se puede ejecutar en cualquier momento y todas las veces que se desee.	2	Inconsistencias: debido al error humano, conducen a desviaciones del estándar de configuración.
3	Control de versiones: donde haya código, también es posible controlar las versiones.	3	Falta de agilidad: se limita limitar la velocidad a la que la organización puede lanzar nuevas versiones de servicios en respuesta a las necesidades e indicadores de mercado.
4	Minimizar costes/esfuerzo: automatizar la administración de la infraestructura ahorra muchos costes y horas de trabajo.	4	Dificultad: en alcanzar y mantener conformidad a la corporación o estándares de la industria debido a la falta de procesos repetibles.

Propiedades clave

- **Atomicidad:**

La atomicidad es la propiedad que asegura que una operación se ha realizado o no, y por lo tanto ante un fallo del sistema no puede quedar a medias. Se dice que una operación es atómica cuando es imposible por menos de otra parte de un sistema encontrar pasos intermedios.

- **Idempotencia:**

En matemática y lógica, la idempotencia es la propiedad para realizar una acción determinada varias veces y aun así conseguir el mismo resultado que se obtendría si se realizase una sola vez.

- **Declaratividad:**

Un lenguaje declarativo es un tipo de lenguaje de programación basado más en las matemáticas y en la lógica que los lenguajes imperativos, más cercanos estos al razonamiento humano. **Los lenguajes declarativos no dicen cómo hacer una cosa, sino, más bien, qué cosa hacer.**

Pilares de AWS Cloudformation

- **Plantillas:**
 - Son susceptibles de ser anidadas e importadas por otras plantillas.
 - El motor de plantillas ofrece funciones auxiliares similares a las de cualquier lenguaje de programación.
 - Permiten definir parámetros y, a través de estos, se pueden escribir plantillas reutilizables para distintos entornos. Se pueden definir condiciones y utilizarlas para crear un recurso u otro en base a una condición.
- **Stacks:**
 - Es la implementación de una plantilla combinada con los parámetros necesarios. Una vez creada, permite comprobar y acceder a la información sobre: qué plantilla se ha usado, qué parámetros tenía, cuáles son los recursos creados y la información de salida que proporciona.



Code infrastructure

Code your infrastructure from scratch with the CloudFormation template language, in either YAML or JSON format, or start from many available sample templates



Amazon S3

Check out your template code locally, or upload it into an S3 bucket



AWS CloudFormation

Use AWS CloudFormation via the browser console, command line tools or APIs to create a stack based on your template code



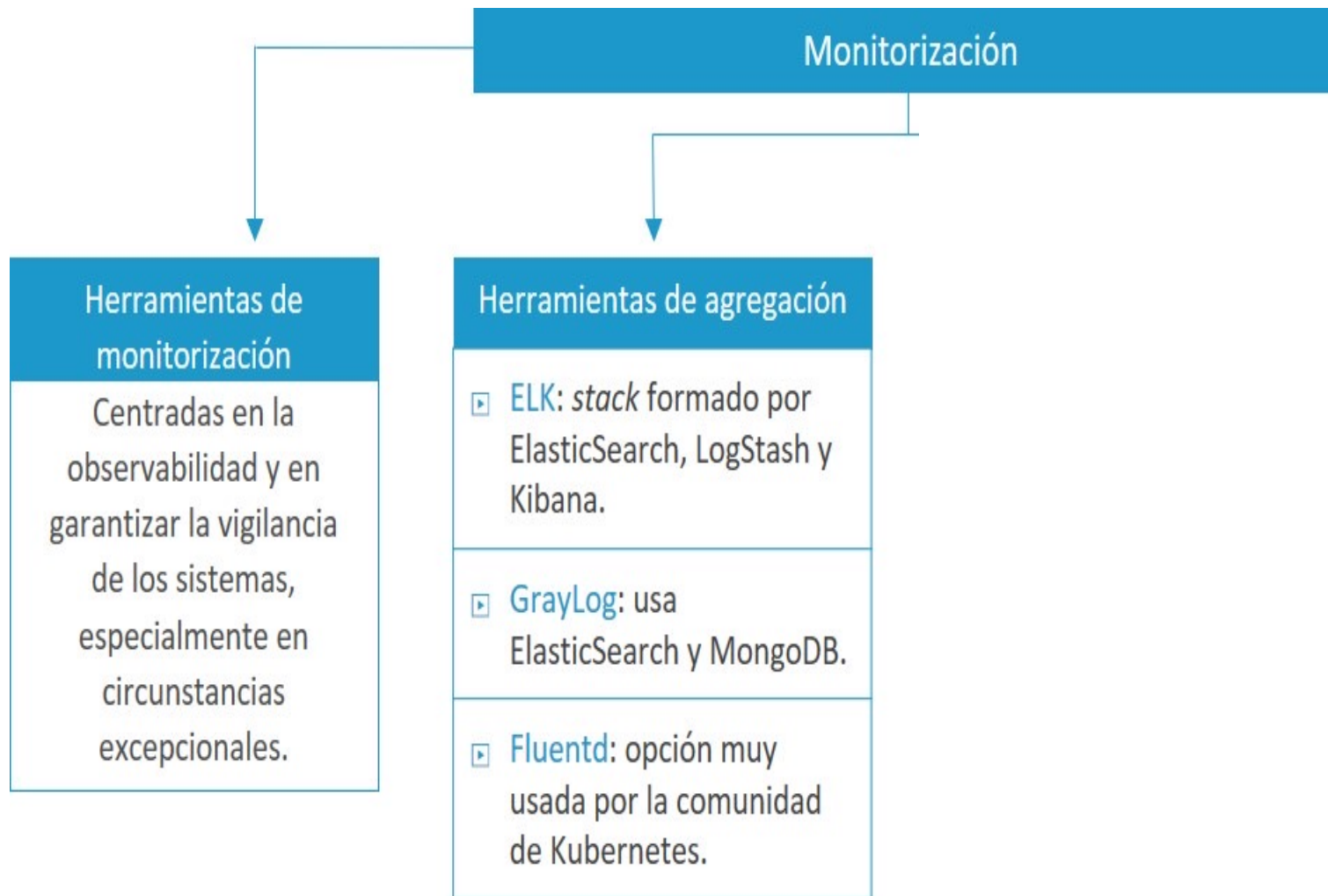
Output

AWS CloudFormation provisions and configures the stacks and resources you specified on your template

```
{
  "AWSTemplateFormatVersion" : "2010-09-09",
  "Description" : "A simple EC2 instance",
  "Resources" : {
    "MyEC2Instance" : {
      "Type" : "AWS::EC2::Instance",
      "Properties" : {
        "ImageId" : "ami-0ff8a91507f77f867",
        "InstanceType" : "t1.micro"
      }
    }
  }
}
```



Tema 9: Monitorización



¿Porque es importante??

If you can not measure it,
you can not improve it.”

— Lord Kelvin

What is measured improves

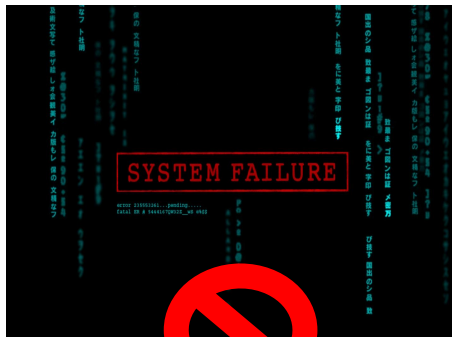
— Peter Drucker

It's not in production unless
it's monitored”.

— Theo Schlossnagle

Porque se necesita monitorización?

Necesitamos asegurar que nuestros servicios y/o aplicaciones:



No **Fallen!**



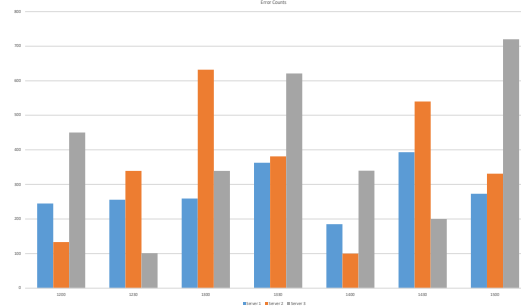
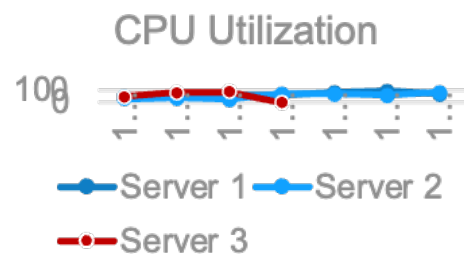
Buena
Experiencia
de usuario

Esto significa:

- La aplicación funciona correctamente.
- Los usuarios obtienen lo que necesitan
- Buenos tiempos de respuesta
- Sin errores

Porque se necesita monitorización? (II)

Es necesario tener indicadores que se puedan medir

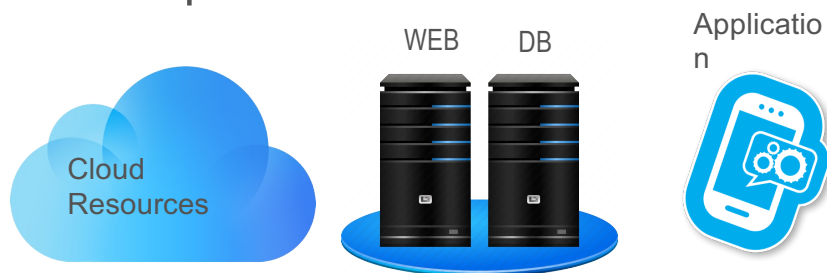


El objetivo es:

Saber el rendimiento de una aplicación

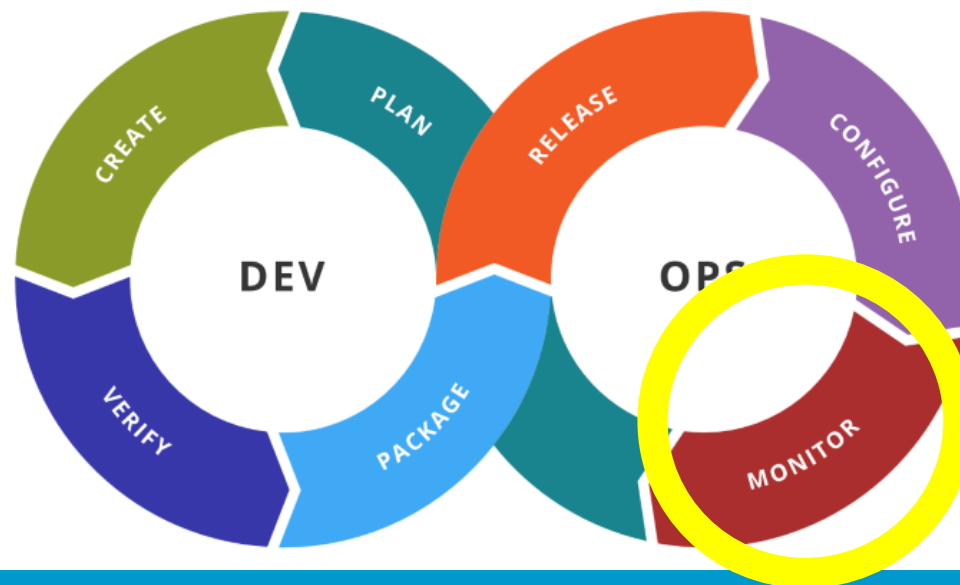
- Recibir alertas si hay algún problema
- Tomar acciones preventivas
- Mejorar la experiencia de usuario
- Ver las tendencias en cuanto al consume de recursos

Medir que?

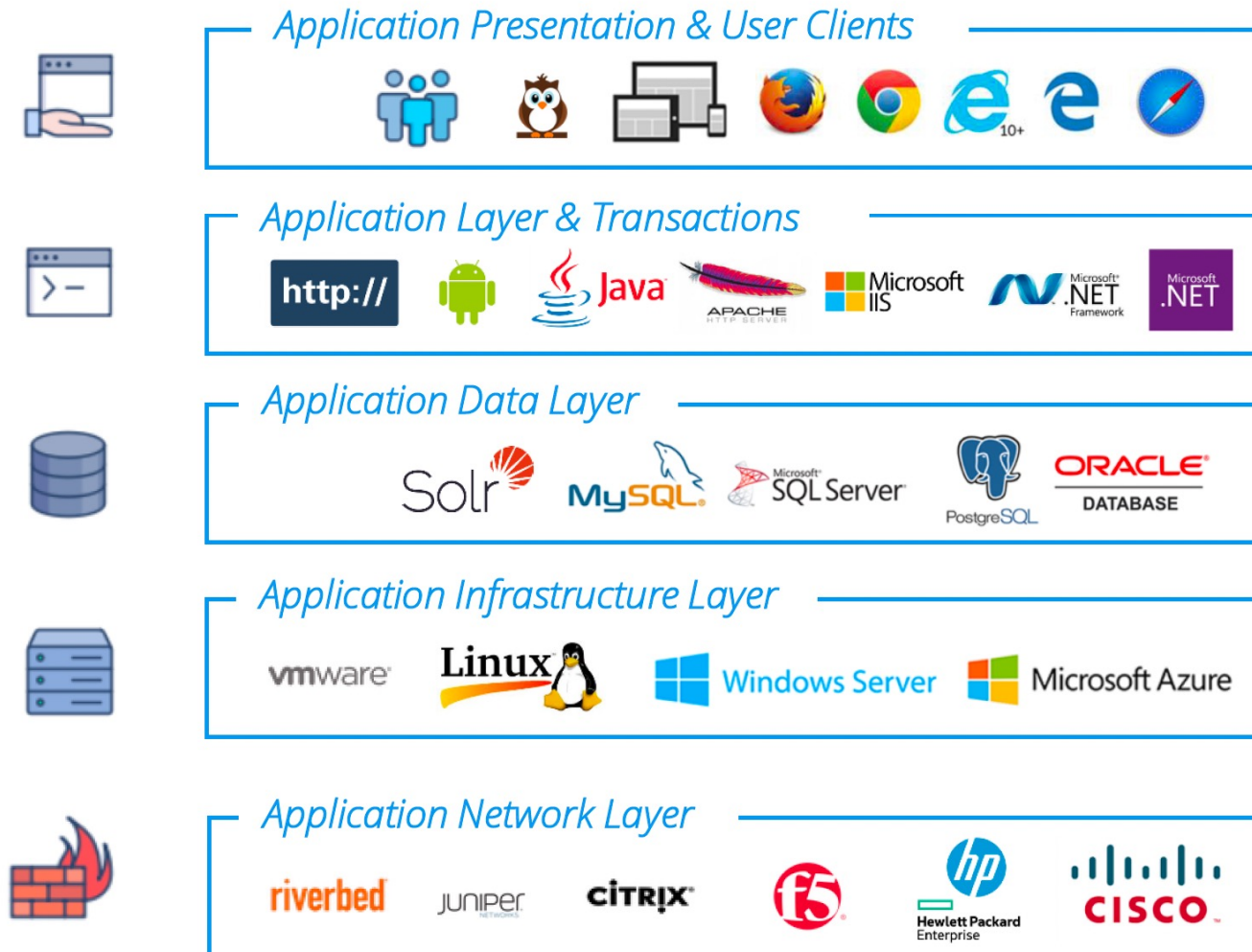


Monitorización y DevOps

- La Monitorización se incluye habitualmente dentro del ciclo de CI/CD que propone DevOps.
- Si uno de nuestros objetivos fundamentales es entregar valor al cliente, es necesario garantizar que las aplicaciones están funcionando para poder estar entregando valor.



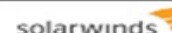
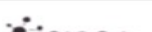
Monitorización de capas



¿Qué aporta la observabilidad?

- Conocer en tiempo real el estado de los diferentes ecosistemas
- Reducir los tiempos de resolución de incidencias.
- Resolver los problemas de forma más rápida y dedicando menos recursos.
- Aportar cuadros de mando de alto y bajo nivel a diferentes departamentos de la empresa.
- Cuadro de mando unificado.
- Capacidad de correlación de información proveniente de distintas fuentes

Herramientas observabilidad

System		Logs		Application		Errors	
Nagios			splunk	APPDYNAMICS	Scout		RAYGUN
Sensu				solarwinds appoptics	INSTANA		
Site24x7	zenoss				New Relic.	OverOps	
			graylog			Synthetic	
							
				Time Series			
							
MONIT	Outlyer	Networks				SMARTBEAR	
	ZABBIX						
					MUNIN	Mobile	
Suites							
IBM			NETSCOUT				
ORACLE		Anomaly					
				Specialized			
						Web	
							

 BigPanda



Tema 10:

Visualización y Alarmas

Cloud Computing, DevOps y Cultura DevOps.

Visualización y alarmas

Herramientas de visualización

Múltiples formas de visualizar y diferentes herramientas para solucionar el problema, con Grafana como la más extendida.

Herramientas de rastreo

Las herramientas de rastreo distribuido permiten el seguimiento de una búsqueda, a través de un sistema de software que se distribuye por múltiples aplicaciones, servicios y bases de datos, así como capas intermedias y otras piezas de software.

Las alarmas son un caso especial

Visualización y alarmas

Las herramientas de visualización también son llamadas herramientas de observabilidad.

Estas últimas provienen de la teoría del control y hacen referencia a nuestra capacidad para entender un sistema a través de sus entradas y salidas.

Podría considerarse a las herramientas de visualización y alerta como aquellas que proporcionan representaciones estructuradas de salidas del sistema.

Las alertas son, básicamente, una selección de salidas negativas del sistema y las visualizaciones son representaciones estructuradas desambiguadas, enfocadas en facilitar la comprensión al usuario.

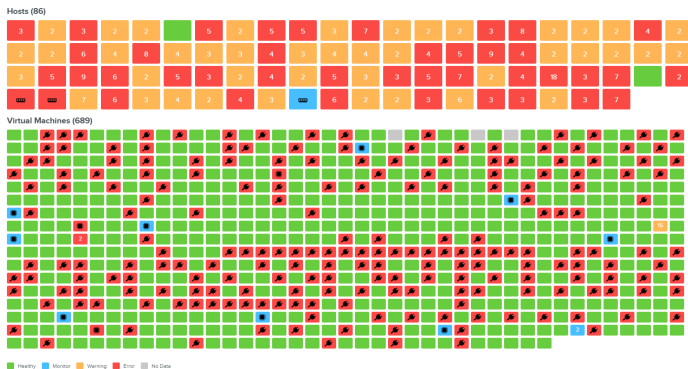
Característica fundamental

- Visualización: La visualización tiene que ser sencilla y comunicar información valiosa.
- Alarmas: Las alertas tienen que ser reducidas y significativas.



Mapas de calor

Este tipo de visualización resulta extremadamente útil para representar la información que proviene de histogramas. Es similar a un gráfico de barras, pero más enriquecido: puede mostrar gradientes dentro de las barras, que representan los diferentes percentiles respecto a la métrica general. Por ejemplo, si necesitas buscar latencias de solicitud y comprender rápidamente la tendencia general, así como la distribución de todas las solicitudes, el mapa de calor resulta un gran aliado. Además, permite usar colores para desambiguar la cantidad de cada sección de las barras.



Mapas de calor

Avg Lead Time for Changes (days)	Avg Speed Of Deployment (min)	Deployment Frequency (#/month)	Mean Time to Restore Service (MTTR) (min)	Change Failure Rate (CFR) (%)	Code Coverage (%)	Functional, Sanity & Regression Pass Rate (%)	Functional, Sanity & Regression Tests Count (#)	Bug Score	Bug Count (#)	SAST Vulns Score	SAST Vulns Count (#)	Dependabot Vulns [Low] Count (#)	Dependabot Vulns [Moderate] Count (#)	Dependabot Vulns [High] Count(#)	Dependabot Vulns [Critical] Count (#)
0.4	5.5	96.8	53.00	10.42	0	No Data	No Data	C	5	A	0	0	10	5	2
0.2	4.8	84.6	53.00	10.42	0	No Data	No Data	A	0	A	0	0	10	8	2
2.7	23.7	61.6	No Service Disruptions	0.00	87.4	No Data	No Data	A	0	A	0	2	6	5	0
0.2	4.8	55.3	53.00	10.42	0	No Data	No Data	C	5	A	0	0	10	5	2
0.2	4.0	51.6	53.00	10.42	0	No Data	No Data	C	63	A	0	0	8	7	2
0.2	4.5	44.2	53.00	10.42	0	No Data	No Data	D	19	A	0	0	10	14	3
0.4	4.9	42.3	53.00	10.42	0	No Data	No Data	C	20	A	0	0	7	4	1
0.2	3.6	37.9	53.00	10.42	0	No Data	No Data	C	63	A	0	0	8	7	2
No Data	8.8	37.5	No Service Disruptions	0.00	100	100.0	6	C	3	A	0	0	0	0	0
1.5	35.8	37.0	No Service Disruptions	0.00	80.5	No Data	No Data	C	22	A	0	3	9	10	0
0.2	4.8	36.4	53.00	10.42	0	No Data	No Data	A	0	A	0	0	0	7	2

Medidores

Los medidores se usan para mostrar una sola métrica de forma rápida, igual que un velocímetro muestra la velocidad, o un medidor de gas representa la cantidad de gas. Al igual que ocurre con un medidor de gas, la mayoría de los medidores de monitorización indican claramente qué es bueno y qué no. El código de colores es el indicador visual que permite al usuario interpretar el estado del sistema: el verde indica estabilidad, el naranja, riesgo inminente, y el rojo, estado crítico.



Kibana

Kibana es un proyecto de código abierto que forma parte del paquete de productos Elastic Stack (ELK). Proporciona capacidades de visualización y exploración de datos indexados sobre el motor de analítica Elasticsearch.

Su principal ventaja es que presenta la información de manera visual a través de dashboards personalizables, mediante la integración de filtros facetados por rangos, categorías de información, cobertura geoespacial o intervalos temporales, entre otros. Además, dispone de un catálogo de herramientas de desarrollo (Dev Tools) para interactuar con los datos almacenados en Elasticsearch.

Grafana

Grafana permite crear visualizaciones interactivas y dinámicas. Originalmente empezó siendo un componente de Kibana, pero en la actualidad son herramientas completamente independientes.

Esta herramienta permite a los usuarios interactuar, intercambiando datos o dashboards. Además, integra diferentes bases de datos como: PostgreSQL, MySQL y Elasticsearch, de las que se puede obtener métricas, filtrar datos o realizar anotaciones en tiempo real.

unir

LA UNIVERSIDAD
EN INTERNET



www.unir.net